

CERTIFICATION STUDY NOTES

Certified Kubernetes Security Specialist

Hands-on labs across the six CKS exam domains

Vijay Vishwakarma

vijay@mevijay.com

Kubernetes v1.34

6 domains · 97 topics

Published 01 September 2026

Contents

Domain 1 · Cluster Setup	5
Configure etcd Binaries	5
Configure Certificate Authority	6
Workflow - Issuance of Signed Certificates	6
etcd - Transport Security with HTTPS	7
Practical - Mutual TLS Authentication	8
Integrating systemd with etcd	9
Configuring API Server	10
Transport Security for API Server	11
Static Token Authentication	13
Downsides - Static Token Authentication	13
Implementing X509 Client Authentication	14
Authorization	15
Encryption Providers	16
Implementing Auditing	17
Setting up a kubeadm Cluster	18
Revising Taints and Tolerations	20
Kubelet Security	22
Verifying Platform Binaries	23
Practical - Ingress with TLS	23
Ingress Annotation - SSL Redirect	25
Structure of a Network Policy	25
Practical - Network Policies	26
Network Policies - Except, Port and Protocol	30
Reference - systemd Unit Files for etcd	31
Reference - kubeadm Installation Steps	32
Reference - Ingress Resource Rules	34
Reference - Ingress Controller and Name-Based Routing	34
Domain 2 · Cluster Hardening	36
Creating a Token for RBAC Practicals	36
Practical - Role and RoleBinding	36
Practical - ClusterRole and ClusterRoleBinding	37
Overview of Service Accounts	38
Service Accounts - Points to Note	39
Service Account Security	39
Setup Environment for Upgrading Clusters	41
Practical - Upgrade Control Plane Node	41
Practical - Upgrade Worker Node	42
Overview of Projected Volumes	44
Mounting Service Accounts using Projected Volumes	45
Reference - Exploring API Groups and RBAC Roles	47
Reference - kubeadm Install with a Specific Version	48
Reference - Deploying an Ingress Controller	50
Domain 3 · Minimize Microservice Vulnerabilities	52
Linux Capabilities	52
Practical - Linux Capabilities	53

Security Contexts	54
Privileged Pods	56
Set Capabilities for Containers	57
ImagePullPolicy	58
Admission Controller - AlwaysPullImages	59
Practical - Pod Security Standards	59
Modes - Pod Security Admission	61
Points to Note - PSA and PSS	61
Practical - ImagePolicyWebhook	62
Practical - Kubernetes Secrets	64
Practical - Configuring Cilium for Kubernetes	65
Structure of Cilium Network Policies	66
Cilium Network Policies - Entities	69
Cilium Network Policies - Layer 4 Rules	71
Cilium Network Policies - DNS Rules	72
Cilium - Deny Policies	72
Cilium - Transparent Encryption using IPsec	74
Cilium - Transparent Encryption using WireGuard	77
Security Context - readOnlyRootFilesystem	79
Practical - Installing Istio	80
Istio - Automatic Sidecar Proxy Injection	81
Practical - Mutual TLS with Istio	81
Reference - HostPath Volumes	83
Reference - Host PID Namespace	84
Reference - Mounting Secrets into Pods	85
Reference - Custom ImagePolicyWebhook Configuration	86
Reference - Cilium Network Policy for Services	87
Reference - Attack Case Study - Reading a Protected Pod	88
Domain 4 · System Hardening	90
Practical - AppArmor	90
AppArmor and Kubernetes	91
Configuring containerd and runc	92
Container Runtime Interface	93
Implementing RuntimeClass - gVisor	94
Reference - Network Policies Practical	96
Reference - Network Policies Practical (Variant)	99
Reference - kubeadm Cluster with Calico	101
Domain 5 · Supply Chain Security	104
Scan Images for Known Vulnerabilities	104
Scanning K8s Clusters for Security Best Practices	104
Static Analysis	105
Dockerfile - Security Best Practices	105
Securing the Docker Daemon	107
Practical - Docker Daemon Socket over TLS	109
SBOM Practical	110
Reference - Running the Docker Daemon Manually	111
Domain 6 · Monitoring, Logging and Runtime Security	113
Installing Falco	113
Falco - Practical	113
Writing Custom Falco Rules and Macros	114
Falco Rule for /dev/mem	115

Falco Configuration File	116
Introduction to Sysdig	117
Audit Logging	117
Reference - Falco Installation from Packages	118
Reference - Custom Falco Rule Examples	119
Reference - Falco from an Exam Perspective	120

Domain 1 · Cluster Setup

Securing the cluster control plane: etcd, TLS everywhere, API server flags, authentication, authorization, encryption at rest, auditing, kubelet security, ingress TLS and network policies.

Configure etcd Binaries

Step 1: Create the Base Binaries Directory

```
mkdir /root/binaries  
  
cd /root/binaries
```

Step 2: Download and Copy the ETCD Binaries to Path

```
wget https://github.com/etcd-io/etcd/releases/download/v3.5.18/etcd-v3.5.18-linux-  
amd64.tar.gz  
  
tar -xzf etcd-v3.5.18-linux-amd64.tar.gz  
  
cd /root/binaries/etcd-v3.5.18-linux-amd64/  
  
cp etcd etcdctl /usr/local/bin/
```

Step 3: Start etcd

```
cd /tmp  
etcd
```

Step 4: Verification - Store and Fetch Data from etcd

```
etcdctl put key1 "value1"
```

```
etcdctl get key1
```

Configure Certificate Authority

Step 1 - Creating a private key for Certificate Authority:

```
mkdir /root/certificates  
  
cd /root/certificates
```

Step 2 - Creating Private Key and CSR:

```
openssl genrsa -out ca.key 2048  
  
openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr
```

Step 3 - Self-Sign the CSR:

```
openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt -days 1000
```

Step 4 - Remove the CSR

```
rm -f ca.csr
```

Step 5 - Check Contents of Certificate

```
openssl x509 -in ca.crt -text -noout
```

Workflow - Issuance of Signed Certificates

Step 1 - Generate Client CSR and Client Key:

```
cd /root/certificates
```

```
openssl genrsa -out client.key 2048  
  
openssl req -new -key client.key -subj "/CN=devuser" -out client.csr
```

Step 2 - Sign the Client CSR with Certificate Authority

```
openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -out client.crt -days 1000
```

Step 3 - Verify Client Certificate

```
openssl x509 -in client.crt -text -noout

openssl verify -CAfile ca.crt client.crt
```

Step 4 - Delete the Client Certificate and Key

```
rm -f client.crt client.key client.csr
```

etcd - Transport Security with HTTPS

Pre-Requisite: Install Network Utilities

```
apt-get -y install tcpdump net-tools
```

Step 1 - Capture Plain Text ETCD Traffic

First tab:

```
cd /tmp

etcd
```

Second tab:

```
tcpdump -i lo -X port 2379
```

Third tab:

```
etcdctl put course "cks"

etcdctl get course
```

Step 2 - Creating the etcd Key:

```
cd /root/certificates

openssl genrsa -out etcd.key 2048
```

Step 3 - Creating Configuration for etcd CSR:

Replace the IP.1 with your Server IP

```
cat > etcd.cnf <<EOF
[req]
req_extensions = v3_req
distinguished_name = req_distinguished_name
```

```
[req_distinguished_name]
[ v3_req ]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names
[alt_names]
IP.1 = [IP-1]
IP.2 = 127.0.0.1
EOF
```

Step 4 - Creating CSR:

```
openssl req -new -key etcd.key -subj "/CN=etcd" -out etcd.csr -config etcd.cnf
```

Step 5 - Sign etcd CSR with Certificate Authority Certificate:

```
openssl x509 -req -in etcd.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
etcd.crt -extensions v3_req -extfile etcd.cnf -days 2000
```

Step 6 - Start Etcd Server with HTTPS:

```
etcd --cert-file=/root/certificates/etcd.crt --key-file=/root/certificates/etcd.key
--advertise-client-urls=https://127.0.0.1:2379 --listen-client-urls=https://
127.0.0.1:2379
```

Step 7 - Verification

Below command should not work.

```
etcdctl put course "cks"
```

Step 8 - Store and Fetch Data by skipping TLS Verification

```
etcdctl --endpoints=https://127.0.0.1:2379 --insecure-skip-tls-verify --insecure-
transport=false put course "cks"

etcdctl --endpoints=https://127.0.0.1:2379 --insecure-skip-tls-verify --insecure-
transport=false get course
```

Practical - Mutual TLS Authentication

Step 1 - Generate Client Certificate and Client Key:

```
cd /root/certificates
```

```
openssl genrsa -out client.key 2048
openssl req -new -key client.key -subj "/CN=client" -out client.csr
openssl x509 -req -in client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
client.crt -extensions v3_req -days 2000
```

Step 2 - Start Etcd Server (Tab 1):

```
cd /root/certificates

etcd --cert-file=etcd.crt --key-file=etcd.key --advertise-client-urls=https://
127.0.0.1:2379 --client-cert-auth --trusted-ca-file=ca.crt --listen-client-
urls=https://127.0.0.1:2379
```

Connect to Etcd (Tab 2)

```
cd /root/certificates

etcdctl --endpoints=https://127.0.0.1:2379 --cacert=ca.crt --cert=client.crt --
key=client.key put key1 "value1"

etcdctl --endpoints=https://127.0.0.1:2379 --cacert=ca.crt --cert=client.crt --
key=client.key get key1
```

Integrating systemd with etcd

Step 1: Create Data Directory for etcd

```
mkdir /var/lib/etcd
chmod 700 /var/lib/etcd
```

Step 2: Create Systemd file for etcd

```
cat <<EOF | sudo tee /etc/systemd/system/etcd.service
[Unit]
Description=etcd
Documentation=https://github.com/coreos

[Service]
ExecStart=/usr/local/bin/etcd \\\
--cert-file=/root/certificates/etcd.crt \\\
--key-file=/root/certificates/etcd.key \\\
--trusted-ca-file=/root/certificates/ca.crt \\\
--client-cert-auth \\\
--listen-client-urls https://127.0.0.1:2379 \\\
--advertise-client-urls https://127.0.0.1:2379 \\\
--data-dir=/var/lib/etcd
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

Step 3: Start etcd

```
systemctl start etcd

systemctl enable etcd
```

Step 4: Verify the status

```
systemctl status etcd
```

Step 5: Check etcd Logs

```
journalctl -u etcd
```

Configuring API Server

Step 1: Download Kubernetes Server Binaries

```

cd /root/binaries

wget https://dl.k8s.io/v1.34.11/kubernetes-server-linux-amd64.tar.gz

tar -xzf kubernetes-server-linux-amd64.tar.gz

ls -lh /root/binaries/kubernetes/server/bin/

cd /root/binaries/kubernetes/server/bin/

cp kube-apiserver kubectrl /usr/local/bin/

```

Step 2 - Generate Client Certificate for API Server (etcd authentication):

```
cd /root/certificates
```

```

{
openssl genrsa -out api-etcd.key 2048

openssl req -new -key api-etcd.key -subj "/CN=kube-apiserver" -out api-etcd.csr

openssl x509 -req -in api-etcd.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out api-etcd.crt -days 2000
}

```

Step 3 - Generate Service Account Certificates

```

{
openssl genrsa -out service-account.key 2048

openssl req -new -key service-account.key -subj "/CN=service-accounts" -out service-account.csr

openssl x509 -req -in service-account.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out service-account.crt -days 100
}

```

Step 4 - Start kube-apiserver:

```
/usr/local/bin/kube-apiserver --advertise-address=<IP-ADDRESS> --etcd-cafile=/root/
certificates/ca.crt --etcd-certfile=/root/certificates/api-etcd.crt --etcd-
keyfile=/root/certificates/api-etcd.key --service-cluster-ip-range
10.0.0.0/24 --service-account-issuer=https://127.0.0.1:6443 --service-
account-key-file=/root/certificates/service-account.crt --service-account-
signing-key-file=/root/certificates/service-account.key --etcd-
servers=https://127.0.0.1:2379
```

Step 5 - Verify

```
netstat -ntlp
curl -k https://localhost:6443
```

Step 6 - Integrate Systemd with API server

Change the IP address in --advertise-address

```
cat <<EOF | sudo tee /etc/systemd/system/kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
ExecStart=/usr/local/bin/kube-apiserver \
--advertise-address=<IP-ADDRESS> \
--etcd-cafile=/root/certificates/ca.crt \
--etcd-certfile=/root/certificates/api-etcd.crt \
--etcd-keyfile=/root/certificates/api-etcd.key \
--etcd-servers=https://127.0.0.1:2379 \
--service-account-key-file=/root/certificates/service-account.crt \
--service-cluster-ip-range=10.0.0.0/24 \
--service-account-signing-key-file=/root/certificates/service-account.key \
--service-account-issuer=https://127.0.0.1:6443

[Install]
WantedBy=multi-user.target
EOF
```

Step 7 - Start kube-api server

```
systemctl start kube-apiserver

systemctl status kube-apiserver
```

Transport Security for API Server

Step 1 - Verify Certificate Details

```
openssl s_client -showcerts -connect localhost:6443 2>/dev/null | openssl x509
-inform pem -noout -text
```

Step 2 - Generate Configuration File for CSR Creation:

```
cd /root/certificates
```

```
cat <<EOF | sudo tee api.conf
[req]
req_extensions = v3_req
distinguished_name = req_distinguished_name
[req_distinguished_name]
[ v3_req ]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names
[alt_names]
DNS.1 = kubernetes
DNS.2 = kubernetes.default
DNS.3 = kubernetes.default.svc
DNS.4 = kubernetes.default.svc.cluster.local
IP.1 = 127.0.0.1
IP.3 = 10.0.0.1
EOF
```

Step 3 - Generate Certificates for API Server

```
{
openssl genrsa -out kube-api.key 2048

openssl req -new -key kube-api.key -subj "/CN=kube-apiserver" -out kube-api.csr
-config api.conf

openssl x509 -req -in kube-api.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
kube-api.crt -extensions v3_req -extfile api.conf -days 2000
}
```

Step 4 - Start kube-apiserver with following flags:

Append following flags in service file for API Server

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--tls-cert-file=/root/certificates/kube-api.crt
--tls-private-key-file=/root/certificates/kube-api.key
```

```
systemctl daemon-reload

systemctl restart kube-apiserver
```

Step 5 - Verification

```
openssl s_client -showcerts -connect localhost:6443 2>/dev/null | openssl x509
-inform pem -noout -text

curl -k https://localhost:6443
```

Static Token Authentication

Format of Static Token File:

```
token,user,uid,"group1,group2,group3"
```

Create a token file with required data:

```
nano /root/token.csv
```

```
Dem0Passw0rd#,bob,01,admins
```

Pass the token auth flag:

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--token-auth-file /root/token.csv
```

```
systemctl daemon-reload  
systemctl restart kube-apiserver
```

Verification:

```
curl -k --header "Authorization: Bearer Dem0Passw0rd#" https://localhost:6443  
  
kubectl get secret --server=https://localhost:6443 --token Dem0Passw0rd# --insecure-skip-tls-verify
```

Testing:

```
kubectl create secret generic my-secret --server=https://localhost:6443 --token  
Dem0Passw0rd# --insecure-skip-tls-verify  
kubectl delete secret my-secret --server=https://localhost:6443 --token  
Dem0Passw0rd# --insecure-skip-tls-verify
```

Downsides - Static Token Authentication

Step 1 Remove Token for BOB user

From the below file, remove the line associated with the Bob user

```
nano /root/token.csv
```

Step 2 - Authenticate with BOB token to API Server

```
curl -k --header "Authorization: Bearer Dem0Passw0rd#" https://localhost:6443
kubectl get secret --server=https://localhost:6443 --token Dem0Passw0rd# --insecure--skip-tls-verify
```

Step 3 - Remove the Static Token File Authentication

```
nano /etc/systemd/system/kube-apiserver.service
```

Remove the --token-auth option

```
systemctl daemon-reload
systemctl restart kube-apiserver
```

Implementing X509 Client Authentication

Documentation Referred:

<https://kubernetes.io/docs/reference/command-line-tools-reference/kube-apiserver/>

Step 1 Creating Certificate for Alice:

```
cd /root/certificates
```

```
openssl genrsa -out alice.key 2048
openssl req -new -key alice.key -subj "/CN=alice/0=developers" -out alice.csr
openssl x509 -req -in alice.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out
alice.crt -days 1000
```

Step 2 Set ClientCA flag in API Server:

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--client-ca-file /root/certificates/ca.crt
```

```
systemctl daemon-reload
systemctl restart kube-apiserver
```

Step 3 Verification:

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/alice.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/alice.key
```

Authorization

Step 1 - Enable AlwaysDeny Authorization Mode

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--authorization-mode=AlwaysDeny
```

```
systemctl daemon-reload
systemctl restart kube-apiserver
```

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/alice.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/alice.key
```

Step 2 - Enable RBAC Authorization Mode

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--authorization-mode=RBAC
```

```
systemctl daemon-reload
systemctl restart kube-apiserver
```

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/alice.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/alice.key
```

Step 3 - Create Super User

```
cd /root/certificates
openssl genrsa -out bob.key 2048
openssl req -new -key bob.key -subj "/CN=bob/O=system:masters" -out bob.csr
openssl x509 -req -in bob.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out bob.crt
-days 1000
```

Step 4 Verification:

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/bob.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/bob.key
```

Encryption Providers

Documentation Referred:

<https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/>

Step 1: Create a new secret

```
kubectl create secret generic new-secret -n default --from-
literal=user=secretpassword --server=https://127.0.0.1:6443 --client-
certificate /root/certificates/bob.crt --certificate-authority /root/
certificates/ca.crt --client-key /root/certificates/bob.key
```

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/bob.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/bob.key
```

Step 2: Find the Secret in ETCD in Plain-Text

```
cd /root/certificates
```

```
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --insecure-skip-tls-verify
--insecure-transport=false --cert ./api-etcd.crt --key ./api-etcd.key get /
registry/secrets/default/new-secret | hexdump -C
```

```
cd /var/lib/etcd
grep -R "secretpassword" .
```

Step 3 - Create Encryption Key:

```
ENCRYPTION_KEY=$(head -c 32 /dev/urandom | base64)
echo $ENCRYPTION_KEY
```

Step 4 - Create Encryption Config:

```
cat > encryption-at-rest.yaml <<EOF
kind: EncryptionConfig
apiVersion: v1
resources:
- resources:
  - secrets
providers:
- aescbc:
  keys:
  - name: key1
    secret: ${ENCRYPTION_KEY}
- identity: {}
EOF
```

Step 5 - Copy configuration to appropriate path:

```
mkdir /var/lib/kubernetes
mv encryption-at-rest.yaml /var/lib/kubernetes
```

Step 6 - Start kube-apiserver with appropriate flag:

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--encryption-provider-config=/var/lib/kubernetes/encryption-at-rest.yaml
```

```
systemctl daemon-reload
systemctl restart kube-apiserver
systemctl status kube-apiserver
```

Step 7 - Create a new Secret

```
kubectl create secret generic db-secret -n default --from-literal=dbadmin=dbpasswd --
server=https://127.0.0.1:6443 --client-certificate /root/certificates/
bob.crt --certificate-authority /root/certificates/ca.crt --client-key /root/
certificates/bob.key
```

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/bob.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/bob.key
```

Step 8: Verify if you can find secret

```
cd /root/certificates
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 --insecure-skip-tls-verify
--insecure-transport=false --cert ./api-etcd.crt --key ./api-etcd.key get /
registry/secrets/default/db-secret | hexdump -C
```

```
cd /var/lib/etcd
grep -R "dbpasswd" .
```

Implementing Auditing

Reference Websites:

<https://jsonformatter.curiousconcept.com/#>

Step 1 - Create Sample Audit Policy File:

```
nano /root/certificates/logging.yaml
```

```
apiVersion: audit.k8s.io/v1
kind: Policy
```

```
rules:
- level: Metadata
```

Step 2 - Audit Configuration:

```
nano /etc/systemd/system/kube-apiserver.service
```

```
--audit-policy-file=/root/certificates/logging.yaml --audit-log-path=/var/log/api-
audit.log --audit-log-maxage=30 --audit-log-maxbackup=10 --audit-log-
maxsize=100
```

```
systemctl daemon-reload
systemctl restart kube-apiserver
```

Step 3 Run Some Queries using Bob user

```
kubectl get secret --server=https://127.0.0.1:6443 --client-certificate /root/
certificates/bob.crt --certificate-authority /root/certificates/ca.crt --
client-key /root/certificates/bob.key
```

Step 4: Verification

```
cd /var/log
grep -i bob api-audit.log
```

Setting up a kubeadm Cluster

Control Plane Node Configuration

Step 1: Setup containerd

```
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

```
modprobe overlay
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
```

```
sysctl --system
```

```
apt-get install -y containerd
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml
```

```
nano /etc/containerd/config.toml
```

```
--> SystemdCgroup = true
```

```
systemctl restart containerd
```

Step 2: Kernel Parameter Configuration

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF
```

```
sudo sysctl --system
```

Step 3: Configuring Repo and Installation

```
sudo apt-get update
apt-get install -y apt-transport-https ca-certificates curl gpg

curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --
    dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
    pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/
    kubernetes.list
```

```
sudo apt-get update
apt-get install -y kubelet=1.34.11-1.1 kubeadm=1.34.11-1.1 kubectl=1.34.11-1.1 cri-
    tools=1.34.0-1.1
sudo apt-mark hold kubelet kubeadm kubectl
systemctl enable --now kubelet
```

Step 4 - Initialize Cluster with kubeadm:

```
kubeadm init --pod-network-cidr=192.168.0.0/16 --kubernetes-version=1.34.11
```

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 5 - Remove the Taint:

```
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

Step 6 - Install Network Addon (Calico):

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/tigera-operator.yaml

kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.29.1/manifests/custom-resources.yaml
```

7 - Verification:

```
kubectl get nodes
kubectl run nginx --image=nginx
kubectl get pods
```

Worker Node Configuration

Run Step 1 to Step 3 from Master Node configuration in worker node as well

Use the `kubeadm join` command that was generated in your Control Plane Node server. The below command is just for reference.

```
kubeadm join <IP-ADDRESS>:6443 --token 9vxoc8.cji5a4o82sd6lkqa \
--discovery-token-ca-cert-hash
sha256:1818dc0a5bad05b378dd3dcec2c048fd798e8f6ff69b396db4f5352b63414baf
```

Run the following command in Mater node to ensure that worker node is in Ready status.

```
kubectl get nodes
```

Revising Taints and Tolerations**Documentation Referenced**

<https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/>

Find Taint for a Node

```
kubectl get nodes

kubectl describe node <node-name>
```

Taint a Node with Effect of NoSchedule

```
kubectl taint node worker-01 key=value:NoSchedule
```

Test with Deployment

```
kubectl create deployment test-deploy --image=nginx --replicas=5

kubectl get pods -o wide
```

Define Tolerations

```
kubectl create deployment test-deploy --image=nginx --replicas=5 --dry-run=client -o
yaml > deployment-toleration.yaml
```

Final File Manifest file after adding toleration

```
apiVersion: apps/v1
kind: Deployment
metadata:
  creationTimestamp: null
  labels:
    app: test-deploy
  name: test-deploy
spec:
  replicas: 5
  selector:
    matchLabels:
      app: test-deploy
  strategy: {}
  template:
    metadata:
      creationTimestamp: null
      labels:
        app: test-deploy
    spec:
      containers:
        - image: nginx
          name: nginx
      tolerations:
        - key: "key"
          operator: "Exists"
          effect: "NoSchedule"
```

```
kubectl create -f deployment-toleration.yaml

kubectl get pods -o wide
```

Taint a Node with Effect of NoExecute

```
kubectl taint node worker-01 key=value:NoExecute

kubectl get pods -o wide
```

Remove Taint from a Node

```
kubectl taint node worker-01 key=value:NoSchedule-
kubectl taint node worker-01 key=value:NoExecute-
```

Kubelet Security

Step 1: Access Worker Node Kubelet from Control Plane Node

```
cd /etc/kubernetes/pki
```

Change the IP address in below command to that of Worker Node IP

```
curl -k --cert apiserver-kubelet-client.crt --key apiserver-kubelet-client.key
https://<IP-ADDRESS>:10250/pods
```

Step 2 Make a request to Kubelet API (Worker Node)

```
apt install net-tools
netstat -ntlp
curl -k -X GET https://localhost:10250/pods
```

Step 2 Modify the Kubelet Configuration file: (Worker Node)

```
cd /var/lib/kubelet
cp config.yaml config.yaml.bak
```

```
Set anonymous auth to false
Authorization Mode to AlwaysAllow
```

```
systemctl restart kubelet
systemctl status kubelet
```

Step 3 Make Request to Kubelet API: (Worker Node)

```
curl -k -X GET https://localhost:10250/pods
```

Step 4 Download kubeletctl: (Worker Node)

GitHub Repo

<https://github.com/cyberark/kubeletctl>

```
wget https://github.com/cyberark/kubeletctl/releases/download/v1.13/
  kubeletctl_linux_amd64 && chmod a+x ./kubeletctl_linux_amd64 && mv ./
  kubeletctl_linux_amd64 /usr/local/bin/kubeletctl
```

```
kubeletctl pods -i
kubeletctl run "whoami" --all-pods -i
```

Step 5: Verify Kubelet Certificate

```
openssl s_client -showcerts -connect 127.0.0.1:10250 2>/dev/null | openssl x509
  -inform pem -noout -text
```

Verifying Platform Binaries

Kubernetes GitHub Repository:

<https://github.com/kubernetes/kubernetes/releases>

Step 1 - Download Binaries: `sh wget https://dl.k8s.io/v1.33.0-alpha.1/kubernetes-client-darwin-arm64.tar.gz`

Step 2 - Verify the Message Digest: `sh sha512sum kubernetes-server-linux-amd64.tar.gz`

Practical - Ingress with TLS

Documentation Referred:

<https://kubernetes.io/docs/concepts/services-networking/ingress/>

Step 1 - Create Basic Pod and Service

```
kubectl run example-pod --image=nginx

kubectl expose pod example-pod --name example-service --port=80 --target-port=80

kubectl get service

kubectl describe service example-service
```

Step 2 - Configure Nginx Ingress Controller

```
kubectl create -f https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/sessions/cks/certified-kubernetes-security-specialist/domain-1-cluster-setup/nginx-controller.yaml

kubectl get pods -n ingress-nginx

kubectl get service -n ingress-nginx
```

Step 3 - Create Self Signed Certificate for Domain:

```
mkdir /root/ingress
cd /root/ingress
openssl req -x509 -nodes -days 365 -newkey rsa:2048 -keyout ingress.key -out
  ingress.crt -subj "/CN=example.internal/0=security"
```

Step 4 - Verify the Default TLS Certificate

Use the NodePort associated with TLS

```
curl -kv <IP>:NodePort
```

Step 5 - Create Kubernetes TLS based secret :

```
kubectl create secret tls tls-certificate --key ingress.key --cert ingress.crt
kubectl get secret tls-certificate -o yaml
```

Step 6 - Create Kubernetes Ingress with TLS:

```
kubectl create ingress demo-ingress --class=nginx --rule=example.internal/*=example-
  service:80,tls=tls-certificate
```

Step 7 - Make a request to Controller:

```
kubectl get service -n ingress-nginx
```

Add the `/etc/hosts` entry for mapping before running this command

```
curl -kv https://example.internal:31893
```

Don't delete the resources created for this practical. We will need it in the next section.

Step 8 - Delete All Resources

ALERT: Don't delete the resources created in this practical. It will be used in the next section.

```
kubectl delete pod nginx-pod
kubectl delete service example-service
kubectl delete ingress demo-ingress
kubectl delete secret tls-certificate

kubectl delete -f https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/
  main/sessions/cks/certified-kubernetes-security-specialist/domain-1-cluster-
  setup/nginx-controller.yaml
```

Ingress Annotation - SSL Redirect

Pre-Requisite:

Complete the setup mentioned in [Practical – Nginx with TLS](#) lecture

Study the default behaviour

```
kubectl get service -n ingress-nginx  
  
curl -kv https://example.internal:<https-node-port>  
  
curl -I http://example.internal:<http-node-port>
```

Modify the Ingress

```
kubectl edit ingress demo-ingress
```

```
metadata:  
  annotations:  
    nginx.ingress.kubernetes.io/ssl-redirect: "false"
```

Test the Setup

```
curl -I http://example.internal:<http-node-port>
```

Structure of a Network Policy

Manifest File Used

first-netpol.yaml

```
apiVersion: networking.k8s.io/v1  
kind: NetworkPolicy  
metadata:  
  name: demo-network-policy  
spec:  
  podSelector:  
    matchLabels:  
      env: production  
  policyTypes:  
    - Ingress  
    - Egress  
  ingress:  
    - from:  
      - podSelector:  
          matchLabels:  
            env: security  
  egress:  
    - to:
```

```
- ipBlock:
  cidr: 8.8.8.8/32
```

```
kubectl create -f first-netpol.yaml

kubectl describe netpol demo-network-policy
```

Delete the Resource created

```
kubectl delete -f first-netpol.yaml
```

Practical - Network Policies

Base Network Policy

base-netpol.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: base-network-policy
```

Example 1

example-1.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-1
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

Create Pod for Testing

```
kubectl run test-pod --image=alpine/curl -- sleep 36000
kubectl get pods
kubectl exec -it test-pod --sh
ping
curl
ping google.com
```

```
kubectl create -f example-1.yaml
kubectl get netpol
kubectl describe netpol base-network-policy
```

Testing the setup

```
kubectl exec -it test-pod --sh
ping google.com
```

```
kubectl delete -f example-1.yaml
```

Example 2

example-2.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-2
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
  - Egress
```

```
kubectl create -f example-2.yaml
kubectl describe netpol base-network-policy
```

Testing the Setup

```
kubectl run random-pod --image=alpine/curl -- sleep 36000
kubectl get pods -o wide
kubectl exec -it random-pod --sh
ping <IP-OF-TEST-POD>
```

```
kubectl create ns testing
kubectl run random-pod -n testing --image=alpine/curl -- sleep 36000
kubectl get pods -n testing
kubectl exec -it random-pod -n testing --sh
ping <IP-OF-TEST-POD>
```

Example 3

example-3.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-3
spec:
  podSelector:
    matchLabels:
      role: suspicious
  policyTypes:
```

- Ingress
- Egress

```
kubectl create -f example-3.yaml
```

Testing the Setup

```
kubectl run suspicious-pod --image=alpine/curl -- sleep 36000
kubectl label pod suspicious-pod role=suspicious
kubectl exec -it suspicious-pod --sh
ping google.com
```

Example 4

example-4.yaml

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-4
spec:
  podSelector:
    matchLabels:
      role: database
  ingress:
    - from:
      - podSelector:
          matchLabels:
            role: app
  policyTypes:
    - Ingress
```

```
kubectl create -f example-4.yaml
```

Testing the Setup

```
kubectl run app-pod --image=alpine/curl -- sleep 36000
kubectl run database-pod --image=alpine/curl -- sleep 36000
kubectl get pods -o wide

kubectl exec -it test-pod --sh
ping <DB-POD-IP>

kubectl label pod app-pod role=app
kubectl exec -it app-pod --sh
ping <DB-POD-IP>
```

```
kubectl delete -f example-4.yaml
```

Example 5

example-5.yaml

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-5
  namespace: production
spec:
  podSelector: {}
  ingress:
    - from:
      - namespaceSelector:
          matchLabels:
            kubernetes.io/metadata.name: security
  policyTypes:
    - Ingress

```

```
kubectl create -f example-5.yaml
```

```

kubectl create ns production
kubectl create ns security

kubectl get ns --show-labels

kubectl run prod-pod -n production --image=alpine/curl -- sleep 36000
kubectl run security-pod -n security --image=alpine/curl -- sleep 36000

kubectl exec -it security-pod -n security --sh
ping <Prod-POD-IP-Of-Production-namespace>

```

```
kubectl delete -f example-5.yaml
```

Example 6

example-6.yaml

```

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: example-6
  namespace: production
spec:
  podSelector: {}
  egress:
    - to:
      - ipBlock:
          cidr: 8.8.8.8/32
  policyTypes:
    - Egress

```

```
kubectl create -f example-6.yaml
```

```

kubectl exec -it prod-pod -n production -- sh
ping 8.8.8.8

```

```
kubectl delete -f example-6.yaml
```

Remove the Created Resources

```
kubectl delete pods --all
kubectl delete pod security-pod -n security
kubectl delete pod prod-pod -n production
kubectl delete pods --all -n testing

kubectl delete ns testing
kubectl delete ns production
kubectl delete ns security
```

Network Policies - Except, Port and Protocol

Example 1 - Except Field

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: except
spec:
  podSelector:
    matchLabels:
      role: database
  ingress:
    - from:
      - ipBlock:
          cidr: 172.17.0.0/16
          except:
            - 172.17.1.0/24
  policyTypes:
    - Ingress
```

```
kubectl create -f except.yaml
```

Example 2 - Port and Protocol

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: multi-port-egress
  namespace: default
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
    - Egress
  egress:
    - to:
      - ipBlock:
          cidr: 10.0.0.0/24
      ports:
```

```
- protocol: TCP
  port: 32000
  endPort: 32768
```

```
kubectl create -f port-proto.yaml
```

Reference - systemd Unit Files for etcd

Create env variable:

```
export SERVER_IP=<IP-ADDRESS>
```

systemd file for etcd:

```
cat <<EOF | sudo tee /etc/systemd/system/etcd.service
[Unit]
Description=etcd
Documentation=https://github.com/coreos

[Service]
ExecStart=/usr/local/bin/etcd \\\
  --cert-file=/root/certificates/etcd.crt \\\
  --key-file=/root/certificates/etcd.key \\\
  --trusted-ca-file=/root/certificates/ca.crt \\\
  --client-cert-auth \\\
  --advertise-client-urls https://127.0.0.1:2379 \\\
  --listen-client-urls=https://127.0.0.1:2379 \\\
  --data-dir=/var/lib/etcd
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
EOF
```

```
systemctl start etcd
systemctl enable etcd
```

systemd file for kube-apiserver:

```
cat <<EOF | sudo tee /etc/systemd/system/kube-apiserver.service
[Unit]
Description=Kubernetes API Server
Documentation=https://github.com/kubernetes/kubernetes

[Service]
ExecStart=/usr/local/bin/kube-apiserver \\\
  --advertise-address=${SERVER_IP} \\\
  --etcd-cafile=/root/certificates/ca.crt \\\
  --etcd-certfile=/root/certificates/apiserver-etcd-client.crt \\\
  --etcd-keyfile=/root/certificates/apiserver-etcd-client.key \\\
  --etcd-servers=https://127.0.0.1:2379 \\\
  --tls-cert-file=/var/lib/kubernetes/kube-api.crt \\\
  --tls-private-key-file=/var/lib/kubernetes/kube-api.key \\\
```

```
--encryption-provider-config=/var/lib/kubernetes/encryption-at-rest.yaml \\  
--v=2  
Restart=on-failure  
RestartSec=5  
  
[Install]  
WantedBy=multi-user.target  
EOF
```

```
systemctl start kube-apiserver  
systemctl enable kube-apiserver
```

Checking Logs:

```
journalctl -u etcd  
journalctl -u kube-apiserver  
  
journalctl -u etcd --since "10 minutes ago"
```

Reference - kubeadm Installation Steps

Documentation Link:

<https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/install-kubeadm/>

Step 1: Setup containerd

```
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf  
overlay  
br_netfilter  
EOF
```

```
modprobe overlay  
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf  
net.bridge.bridge-nf-call-iptables = 1  
net.ipv4.ip_forward = 1  
net.bridge.bridge-nf-call-ip6tables = 1  
EOF
```

```
sysctl --system
```

```
apt-get install -y containerd  
mkdir -p /etc/containerd  
containerd config default > /etc/containerd/config.toml
```

```
nano /etc/containerd/config.toml
```

--> SystemdCgroup = true

```
systemctl restart containerd
```

Step 2: Kernel Parameter Configuration

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF
```

```
sudo sysctl --system
```

Step 3: Configuring Repo and Installation

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.24/deb/Release.key | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:/v1.24/deb/ /' | sudo tee /etc/apt/sources.list.d/
kubernetes.list
```

```
sudo apt-get update
apt-cache madison kubeadm
sudo apt-get install -y kubelet=1.24.2-1.1 kubeadm=1.24.2-1.1 kubectl=1.24.2-1.1 cri-
tools=1.24.2-1.1
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 4: Initialize Cluster with kubeadm (Only master node)

```
kubeadm init --pod-network-cidr=10.244.0.0/16 --kubernetes-version=1.24.2
```

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 5: Install Network Addon (flannel) (master node)

```
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/
Documentation/kube-flannel.yml
```

Location of Manifest files for K8s components:

```
cd /etc/kubernetes/manifests
```

Reference - Ingress Resource Rules

Documentation Referred:

<https://kubernetes.io/docs/concepts/services-networking/ingress/#name-based-virtual-hosting>

Ingress Resource - Rule 1

```
kubectl create ingress --help

kubectl first ingress first-ingress --rule="example.internal/*=example-service:80"

kubectl describe ingress first-ingress
```

Ingress Resource - Rule 2

```
kubectl create ingress second-ingress --rule="example.internal/*=example-service:80"
--rule="demo.internal/*=demo-service:80"
```

Generating Manifest File for Ingress Resource

```
kubectl create ingress second-ingress --rule="example.internal/*=example-service:80"
--rule="demo.internal/*=demo-service:80" --dry-run=client -o yaml
```

Reference - Ingress Controller and Name-Based Routing

Documentation Referred:

<https://kubernetes.io/docs/concepts/services-networking/ingress-controllers/>

<https://kubernetes.github.io/ingress-nginx/deploy/>

Step 1 - Create 2 Pods for Example Service and Demo Service

```
kubectl run example-pod --image=nginx
kubectl run demo-pod --image=httpd
```

Step 2 - Create Service for Both the pods

```
kubectl expose pod example-pod --name example-service --port=80 --target-port=80

kubectl expose pod demo-pod --name demo-service --port=80 --target-port=80

kubectl get service
```

Step 3 - Create Ingress Resource with 2 Rules

```
kubectl create ingress main-ingress --class=nginx --rule="example.internal/*=example-service:80" --rule="demo.internal/*=demo-service:80"

kubectl describe ingress main-ingress
```

Step 4: Install Nginx Ingress Controller:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0/deploy/static/provider/cloud/deploy.yaml
```

Step 5: Verify the Ingress Controller Resource

```
kubectl get pods -n ingress-nginx
kubectl get service -n ingress-nginx
```

Verify if a new load balancer is created in Digital Ocean.

Step 6: Verify the Setup

```
curl -H "Host: example.internal" <LB-IP>
curl -H "Host: demo.internal" <LB-IP>
```

Step 7: Delete All Resource

```
kubectl delete -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.12.0/deploy/static/provider/cloud/deploy.yaml

kubectl delete ingress main-ingress

kubectl delete service example-service
kubectl delete service demo-service
kubectl delete pod example-pod
kubectl delete pod demo-pod
```

Domain 2 · Cluster Hardening

Restricting who can do what in the cluster: RBAC roles and bindings, service account hardening, projected volumes and keeping the cluster patched through kubeadm upgrades.

Creating a Token for RBAC Practicals

Websites Referred:

<https://fusionauth.io/dev-tools/jwt-decoder>

<https://www.unixtimestamp.com/>

Generate Token

```
kubectl create token --duration=100h
```

Practical - Role and RoleBinding

Documentation / Websites Referenced:

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.34/>

Check Resources as part of specific API Group

```
kubectl api-resources --api-group="apps"  
kubectl api-resources --api-group=""
```

Create Role

```
kubectl create role --help  
  
kubectl create role pod-reader --verb=list --resource=pods  
  
kubectl get roles  
  
kubectl describe role pod-reader
```

Test the Permissions

```
kubectl cluster-info
```

Replace the `K8S-DETAILS-HERE` with the output you get in previous command

```
curl -k https://K8S-DETAILS-HERE/api/v1/namespaces/default/pods --header
"Authorization: Bearer $TOKEN"
```

Create RoleBinding

```
kubectl create rolebinding --help

kubectl create rolebinding test-rolebinding --role=pod-reader --
user=system:serviceaccount:default:test-sa

kubectl get rolebinding

kubectl describe rolebinding test-rolebinding
```

Test the Permissions

```
kubectl run test-pod --image=nginx
```

Replace the URL in below command to your K8s URL. If using macOS or Linux, use `$TOKEN` instead of `%TOKEN%`

```
curl -k https://38140ecd-e8d7-4fff-be52-24629c40cdac.k8s.ondigitalocean.com/api/v1/
namespaces/default/pods --header "Authorization: Bearer %TOKEN%"
```

Generate Manifest Files for Role and Role Binding

```
kubectl create role test-role --verb=list --resource=pods --dry-run=client -o yaml

kubectl create rolebinding test-rolebinding --role=pod-reader --
user=system:serviceaccount:default:test-sa --dry-run=client -o yaml
```

Delete Role and RoleBinding

```
kubectl delete role pod-reader

kubectl delete rolebinding test-rolebinding
```

Practical - ClusterRole and ClusterRoleBinding

Create ClusterRole

```
kubectl create clusterrole pod-reader --verb=get,list,watch --resource=pods

kubectl describe clusterrole pod-reader
```

Create ClusterRoleBinding

```
kubectl create clusterrolebinding test-clusterrole --clusterrole=pod-reader --
    user=system:serviceaccount:default:test-sa

kubectl describe clusterrolebinding test-clusterrole
```

Test the Setup

Replace the URL in below command to your K8s URL. If using macOS or Linux, use `$TOKEN` instead of `%TOKEN%`

```
curl -k https://38140ecd-e8d7-4fff-be52-24629c40cdac.k8s.ondigitalocean.com/api/v1/
    namespaces/default/pods --header "Authorization: Bearer %TOKEN%"
```

```
curl -k https://38140ecd-e8d7-4fff-be52-24629c40cdac.k8s.ondigitalocean.com/api/v1/
    namespaces/kube-system/pods --header "Authorization: Bearer %TOKEN%"
```

Overview of Service Accounts

List service account in all namespaces

```
kubectl get serviceaccount --all-namespaces
```

Create a Test App Pod

```
kubectl run app-pod --image=nginx
kubectl get pods
```

Verify Namespace associated with Pod

```
kubectl describe pod app-pod
```

Verify Mounted Token in Pod

```
kubectl exec -it app-pod -- bash

cd /var/run/secrets/kubernetes.io/serviceaccount/

ls

cat token
```

Connect to Kubernetes Cluster using Token

```
token=$(cat token)
echo $token

kubectl cluster-info (from outside of Pod)
```

```
curl -k -H "Authorization: Bearer $token" https://control-plane-url-here/api/v1
```

Service Accounts - Points to Note

Verify Default Service Account In Namespace

```
kubectl get serviceaccount
kubectl get sa
kubectl get sa -n kube-system
```

Create New Namespace

```
kubectl create namespace demo-test
```

Verify if Default Service Account Gets Created in Namespace

```
kubectl get sa -n demo-test
```

Create New Pod in Newly Created Namespace

```
kubectl run nginx-pod -n demo-test --image=nginx
kubectl get pods -n demo-test
kubectl describe pod nginx-pod -n demo-test
```

Verify if Mounted Token is Available inside Pod

```
kubectl exec -it nginx-pod -n demo-test -- bash
cd /var/run/secrets/kubernetes.io/serviceaccount/
ls
cat token
```

Service Account Security

Documentation Referred:

<https://kubernetes.io/docs/tasks/configure-pod-container/configure-service-account/>

Authenticating with Token:

```
kubectl run pod-1 --image=nginx
kubectl exec -it pod-1 -- bash
```

```
cat /var/run/secrets/kubernetes.io/serviceaccount/token
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
```

```
echo $TOKEN
curl -k -H "Authorization: Bearer $TOKEN" "https://kubernetes/api/v1"

curl -k -H "Authorization: Bearer $TOKEN" "https://kubernetes/api/v1/namespaces"
```

Approach 1 - Opt out AutoMounting at Service Account level (Reference):

```
kubectl edit sa default
```

Add following snippet:

```
automountServiceAccountToken: false
```

Verification

```
kubectl run pod-2 --image=nginx

kubectl exec -it pod-2 -- bash

cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Approach 2 Opt out AutoMounting at Pod level

```
nano pod-3.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: demo-pod
spec:
  automountServiceAccountToken: false
  containers:
  - image: nginx
    name: demo-pod
```

```
kubectl apply -f pod-3.yaml
```

Verification Stage:

```
kubectl exec -it demo-pod -- bash

cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Delete the resources created

```
kubectl delete pods --all
```

Setup Environment for Upgrading Clusters

Configure Kubeadm for Control Plane Node

```
curl -L https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/sessions/cks/certified-kubernetes-security-specialist/domain-2-cluster-hardening/kubeadm-automate.sh -o kubeadm-control-plane.sh

chmod +x kubeadm-control-plane.sh

./kubeadm-control-plane.sh
```

Configure Kubeadm for Worker Node (Run in Worker Node)

```
curl -L https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/sessions/cks/certified-kubernetes-security-specialist/domain-2-cluster-hardening/kubeadm-worker-automate.sh -o kubeadm-worker.sh

chmod +x kubeadm-worker.sh

./kubeadm-worker.sh
```

Use the `kubeadm join` command that was generated in your Control Plane Node server. The below command is just for reference.

```
kubeadm join <IP-ADDRESS>:6443 --token 9vxoc8.cji5a4o82sd6lkqa \
--discovery-token-ca-cert-hash
sha256:1818dc0a5bad05b378dd3dcec2c048fd798e8f6ff69b396db4f5352b63414baf
```

Practical - Upgrade Control Plane Node

Documentation Referenced:

<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>

Verify Current Version of kubeadm

```
kubeadm version
```

Remove hold on kubernetes component for upgrade process

```
apt-mark unhold kubeadm kubelet kubectl
```

Configure Repository for newer Kubernetes version

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

```
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
      pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/
      kubernetes.list

apt-get update
```

Identify available Kubeadm version

```
apt-cache madison kubeadm
```

Install and Upgrade Kubeadm

```
apt-get install -y kubeadm="1.34.11-1.1*"
kubeadm upgrade plan
kubeadm upgrade apply v1.34.11
```

Install and Upgrade kubelet and kubectl

```
apt-cache madison kubelet
apt-cache madison kubectl
apt-get install -y kubelet="1.34.11-1.1*" kubectl="1.34.11-1.1"

systemctl daemon-reload
systemctl restart kubelet
```

Mark hold Kubernetes Component

```
apt-mark hold kubelet kubectl kubeadm
```

Final Verfication

```
kubectl get nodes
```

Practical - Upgrade Worker Node

Documentation Referenced:

<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>

Configure Repository for newer Kubernetes version

```
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.34/deb/Release.key | sudo gpg --
      dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
      pkgs.k8s.io/core:/stable:/v1.34/deb/ /' | sudo tee /etc/apt/sources.list.d/
      kubernetes.list
```

```
apt-get update
```

Identify available Kubeadm version

```
apt-cache madison kubeadm
```

Remove hold on kubernetes component for upgrade process

```
apt-mark unhold kubeadm kubelet kubectl
```

Install and Upgrade Kubeadm

```
apt-get install -y kubeadm="1.34.11-1.1*"
kubeadm upgrade node
```

Install and Upgrade kubelet and kubectl

```
apt-cache madison kubelet
apt-cache madison kubectl
```

RUN the below command CONTROL PLANE NODE. Replace `node01` with appropriate worker node name

```
kubectl drain node01 --ignore-daemonsets
```

Following commands will run on worker node

```
apt-get install -y kubelet="1.34.11-1.1*" kubectl="1.34.11-1.1"
sudo systemctl daemon-reload
sudo systemctl restart kubelet
```

Verification: (Run in Control Plane Node)

RUN the below command CONTROL PLANE NODE. Replace `node01` with appropriate worker node name

```
kubectl uncordon node01
kubectl get nodes
```

Overview of Projected Volumes

Documentation Referred:

<https://kubernetes.io/docs/concepts/storage/projected-volumes/>

Create Secret and ConfigMap

```
kubectl create secret generic firstsecret --from-literal=dbpassword=mypassword123

kubectl create configmap my-config --from-literal=config-key="This is a config value"
```

Create Pod with Projected Volume

```
apiVersion: v1
kind: Pod
metadata:
  name: volume-test
spec:
  containers:
    - name: container-test
      image: busybox:1.37
      command: ["sleep", "3600"]
      volumeMounts:
        - name: all-in-one
          mountPath: "/projected-volume"
          readOnly: true
  volumes:
    - name: all-in-one
      projected:
        sources:
          - secret:
              name: firstsecret
              items:
                - key: dbpassword
                  path: my-username
          - configMap:
              name: my-config
              items:
                - key: config-key
                  path: my-config
```

```
kubectl delete -f projected-volume.yaml
```

```
kubectl exec -it volume-test -- sh

cd /projected-volume
```

Mounting Service Accounts using Projected Volumes

Documentation Referred:

<https://kubernetes.io/docs/concepts/storage/projected-volumes/#serviceaccounttoken>

Create ServiceAccount with Automount False

```
kubectl create namespace test-ns  
  
kubectl edit sa default -n test-ns
```

Add following snippet:

```
automountServiceAccountToken: false
```

Use Projected Volume to Mount Service Account to Pod

```
nano sa-example-1.yaml
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: pod-sa  
  namespace: test-ns  
spec:  
  containers:  
    - name: container-test  
      image: busybox:1.37  
      command: ["sleep", "3600"]  
      volumeMounts:  
        - name: token-vol  
          mountPath: "/service-account"  
          readOnly: true  
  volumes:  
    - name: token-vol  
      projected:  
        sources:  
          - serviceAccountToken:  
              audience: api  
              expirationSeconds: 3600  
              path: token
```

```
kubectl create sa-example-1.yaml
```

Verification

```
kubectl exec -n test-ns -it pod-sa -- sh  
ls -l /service-account  
cat /service-account/token
```

```
kubectl delete -f sa-example-1.yaml
```

Mount Custom Service Account

```
nano custom-sa.yaml
```

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: custom-sa
  namespace: test-ns
automountServiceAccountToken: false
```

```
kubectl create -f custom-sa.yaml
```

```
nano sa-example-2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-sa-custom
  namespace: test-ns
spec:
  serviceAccountName: custom-sa
  containers:
  - name: container-test
    image: busybox:1.37
    command: ["sleep", "3600"]
    volumeMounts:
    - name: token-vol
      mountPath: "/service-account"
      readOnly: true
  volumes:
  - name: token-vol
    projected:
      sources:
      - serviceAccountToken:
          audience: api
          expirationSeconds: 3600
          path: token
```

```
kubectl create -f sa-example-2.yaml
```

Verification

```
kubectl exec -n test-ns -it pod-sa-custom -- sh
ls -l /service-account
cat /service-account/token
```

Delet the Resources created

```
kubectl delete ns test-ns
```

Reference - Exploring API Groups and RBAC Roles

Explore the API Groups:

```
kubectl proxy --port 8085
```

```
curl localhost:8085
curl localhost:8085/api/v1
```

Step 1 Create RBAC Role:

```
nano rbac.yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "watch", "list", "create"]
- apiGroups: [""]
  resources: ["pods/exec"]
  verbs: ["create"]
```

```
kubectl apply -f rbac.yaml
```

```
kubectl get role
kubectl describe role pod-reader
```

Step 2 Create Role Binding:

```
nano rolebinding.yaml
```

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-pods
  namespace: default
subjects:
- kind: User
  name: john
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
```

```
name: pod-reader
apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f rolebinding.yaml
kubectl get rolebinding
kubectl describe rolebinding read-pods
```

Step 3 - Verify from user John:

```
su - john
kubectl get pods
kubectl run john-pod --image=nginx
kubectl exec -it john-pod -- bash
```

Step 4 Verify API Access Action:

```
kubectl auth can-i create deployments
kubectl auth can-i create pods
```

Reference - kubeadm Install with a Specific Version

Documentation Link:

<https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/>

Step 1: Setup containerd

```
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

```
modprobe overlay
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
```

```
sysctl --system
```

```
apt-get install -y containerd
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml
```

```
nano /etc/containerd/config.toml
```

--> SystemdCgroup = true

```
systemctl restart containerd
```

Step 2: Kernel Parameter Configuration

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF
```

```
sudo sysctl --system
```

Step 3: Configuring Repo and Installation

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
sudo curl -fsSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
echo "deb [signed-by=/usr/share/keyrings/kubernetes-archive-keyring.gpg] https://apt.kubernetes.io/ kubernetes-xenial main" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

```
sudo apt-get update
apt-cache madison kubeadm
sudo apt-get install -y kubelet=1.22.0-00 kubeadm=1.22.0-00 kubectl=1.22.0-00
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 4: Initialize Cluster with kubeadm (Only master node)

```
kubeadm init --pod-network-cidr=10.244.0.0/16
```

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 5: Install Network Addon (flannel) (master node)

```
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

Step 6: Upgrade Kubeadm Cluster

```
apt-cache madison kubeadm
apt-mark unhold kubelet kubectl kubeadm
apt-get update && apt-get install -y kubelet=1.23.7-00 kubectl=1.23.7-00
kubeadm=1.23.7-00
apt-mark hold kubelet kubectl
```

```
kubeadm version
kubeadm upgrade plan
kubeadm upgrade apply 1.23.0
systemctl daemon-reload
systemctl restart kubelet
```

Reference - Deploying an Ingress Controller

Step 1 - Create Nginx Pod:

```
kubectl run example-pod --image=nginx -l=app=nginx
```

Step 2 - Create Service:

```
kubectl expose example-pod nginx --port=80 --target-port=80 --name=example-svc
```

Step 3 - Installing Ingress Controller:

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/
controller-v1.8.2/deploy/static/provider/baremetal/deploy.yaml
```

Step 4 - Create Ingress:

```
nano ingress.yaml
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: example-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: example.internal
    http:
      paths:
      - pathType: Prefix
        path: "/"
        backend:
          service:
            name: example-svc
            port:
              number: 80
```

```
kubectl apply -f ingress.yaml
```

Step 5 - Verification:

```
kubectl get svc -n ingress-nginx  
apt install net-tools  
ifconfig eth0
```

Add appropriate host entry accordingly.

```
nano /etc/hosts
```

```
curl http://example.internal:30429
```

Domain 3 · Minimize Microservice Vulnerabilities

Hardening the workload itself: Linux capabilities, security contexts, Pod Security Standards, image admission control, secrets, Cilium network policies and mTLS with Istio.

Linux Capabilities

Check Man page for Linux Capabilities

```
man capabilities
```

Verify capability of ping utility

```
which ping  
  
getcap /usr/bin/ping
```

Create new user in Linux

```
useradd -m -s /bin/bash demouser  
su - demouser  
ping google.com
```

Remove capabilities for Ping utility (From Root User)

```
setcap -r /usr/bin/ping  
  
getcap /usr/bin/ping
```

Set the capabilities for ping utility

```
setcap cap_net_raw=ep /usr/bin/ping
```

Practical - Linux Capabilities

Base Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>

#define PORT 900

int main() {
    int server_fd;
    struct sockaddr_in server_addr;

    // Create socket
    server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd < 0) {
        perror("Socket failed");
        exit(EXIT_FAILURE);
    }

    // Set up server address structure
    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(PORT);

    // Bind socket to port 900
    if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Bind failed");
        close(server_fd);
        exit(EXIT_FAILURE);
    }

    printf("Successfully bound to port %d\n", PORT);

    // Close socket
    close(server_fd);

    return 0;
}
```

Install GCC

```
apt-get -y install gcc
```

Compile the Program

```
gcc bind_port_900.c -o bind_port_900
```

Try running as a Normal user

```
./bind_port_900
```

Grant Capability to the Executable

```
setcap 'cap_net_bind_service=+ep' ./bind_port_900
```

Try running as a Normal user

```
./bind_port_900
```

Security Contexts

Documentation Referenced:

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

Example 1 - Insecure Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: insecure-pod
spec:
  containers:
  - name: demo-container
    image: busybox:latest
    command: ["sleep", "36000"]
    volumeMounts:
    - name: host-root
      mountPath: /host
  volumes:
  - name: host-root
    hostPath:
      path: /
```

```
kubectl apply -f pod.yaml
```

```
kubectl get pods
```

```
kubectl exec -it insecure-pod -- sh
id
cd /host
ls
cd /boot
ls -l
```

```
kubectl delete pod insecure-pod
```

Example - Controlled Pod

```
pod-controlled.yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: controlled-pod
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 2000
    fsGroup: 3000
  containers:
  - name: demo-container
    image: busybox:latest
    command: ["sleep", "36000"]
    volumeMounts:
    - name: host-root
      mountPath: /host
  volumes:
  - name: host-root
    hostPath:
      path: /

```

```
kubectl apply -f pod-controlled.yaml
```

```

kubectl get pods
kubectl exec -it controlled-pod -- sh
id
cd /host
ls
cd /boot
cd grub
vi grub.cfg

```

Verify UID, GID, fsGroup

```

cd /host/tmp
touch test.txt
ls -ls

```

Example 3 - fsGroup Pod

```
pod-fs.yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: fsgroup-pod
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 2000
    fsGroup: 3000
  volumes:
  - name: host-root
    emptyDir: {}
  containers:
  - name: demo-container

```

```
image: busybox:latest
command: ["sleep", "36000"]
volumeMounts:
  - name: host-root
    mountPath: /host
```

```
kubectl apply -f pod-fs.yaml
```

Verify UID, GID, fsGroup

```
kubectl exec -it fsgroup-pod -- sh
id
cd /host
touch test.txt
ls -l
```

Delete all the Pods after practical

```
kubectl delete pods --all
```

Privileged Pods

Documentation Referenced:

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

Create Non-Privileged Pod

```
kubectl run non-privileged --image=busybox -- sleep 36000
```

Create Privileged Pod

```
kubectl run privileged-pod --image=busybox --privileged -- sleep 36000
```

Connect to Both the Pods in Two Separate Terminal Tabs

```
kubectl exec -it non-privileged -- sh

kubectl exec -it privileged-pod -- sh
```

Run Commands in Both Pods to See Difference

```
cd /dev
ls

dmesg
```

```
fdisk -l
```

Verify Manifest File Associated with Privileged Pod

```
kubectl run privileged-pod --image=busybox --dry-run=client -o yaml --privileged --
sleep 36000
```

Set Capabilities for Containers

Documentation Referenced:

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/#set-capabilities-for-a-container>

Create Normal Pod

```
kubectl run normal-pod --image=busybox -- sleep 36000

kubectl exec -it normal-pod -- sh
```

Check Capabilities

```
cat /proc/1/status

capsh --decode=<capabilities-here>
```

Capability 1 Pod

```
capability-1.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: capabilities-pod-1
spec:
  containers:
  - name: demo
    image: busybox
    command: ["sleep", "36000"]
    securityContext:
      capabilities:
        add: ["NET_ADMIN", "SYS_TIME"]
```

```
kubectl apply -f capability-1.yaml

kubectl exec -it capabilities-pod-1 -- sh
```

```
cat /proc/1/status

capsh --decode=<capabilities-here>
```

Capability 2 Pod

```
capability-2.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: capabilities-pod-2
spec:
  containers:
    - name: demo-2
      image: busybox
      command: ["sleep", "36000"]
      securityContext:
        capabilities:
          add: ["NET_ADMIN", "SYS_TIME"]
          drop:
            - ALL
```

```
kubectl apply -f capability-2.yaml

kubectl exec -it capabilities-pod-2 -- sh
```

```
cat /proc/1/status

capsh --decode=<capabilities-here>
```

ImagePullPolicy

Using the --help to find exact flag

```
kubectl run --help
```

Create Pod with ImagePullPolicy of Always

```
kubectl run always-pod --image=nginx --image-pull-policy=Always

kubectl describe pod always-pod
```

Create Pod with ImagePullPolicy of IfNotPresent

```
kubectl run if-not-present --image=nginx --image-pull-policy=IfNotPresent

kubectl describe pod if-not-present
```

Create Pod with ImagePullPolicy of Never

```
kubectl run never-pod --image=ubuntu --image-pull-policy=Never

kubectl get pods

kubectl describe pod never-pod
```

Miscellaneous Command used for Demonstration

```
cricctl images
cricctl images -v
```

Admission Controller - AlwaysPullImages

Test the Base Setup (Without AlwaysPullImages Enabled)

```
kubectl run never-pod --image=nginx --image-pull-policy=Never

kubectl describe pod never-pod

kubectl get pod never-pod -o yaml
```

Enable the Admission Controller of AlwaysPullImages:

```
nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

Add following within kube-apiserver manifest to enable admission controller

```
AlwaysPullImages
```

Test the Setup

```
kubectl run never-pod-2 --image=nginx --image-pull-policy=Never

kubectl describe pod never-pod-2

kubectl get pod never-pod -o yaml
```

Practical - Pod Security Standards

Documentation Refernced:

<https://kubernetes.io/docs/reference/generated/kubernetes-api/v1.34/>

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

Create 3 Namespace

```
kubectl create ns privileged-ns
kubectl create ns baseline-ns
kubectl create ns restricted-ns
```

Associate 3 Policy Levels for Namespaces

```
kubectl label namespace privileged-ns pod-security.kubernetes.io/enforce=privileged

kubectl label namespace baseline-ns pod-security.kubernetes.io/enforce=baseline

kubectl label namespace restricted-ns pod-security.kubernetes.io/enforce=restricted
```

Testing

1 - Privileged Pod

```
kubectl run privileged-pod --image=nginx --privileged -n privileged-ns

kubectl run privileged-pod --image=nginx --privileged -n baseline-ns

kubectl run privileged-pod --image=nginx --privileged -n restricted-ns
```

2 - Default Config Pod

```
kubectl run normal-pod --image=nginx -n privileged-ns

kubectl run normal-pod --image=nginx -n baseline-ns

kubectl run normal-pod --image=nginx -n restricted-ns
```

3 - Pod Manifest for Restricted Namespace

nano restrcited-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: restricted-pod
  namespace: restricted-ns
spec:
  containers:
    - name: secure-container
      image: busybox
      command: ["sleep", "3600"]
      securityContext:
        allowPrivilegeEscalation: false
        runAsNonRoot: true
        runAsUser: 1001
      capabilities:
        drop: ["ALL"]
      seccompProfile:
        type: RuntimeDefault
```

```
kubectl create -f restrcited-pod.yaml

kubectl get pods -n restricted-ns

kubectl delete -f restrcited-pod.yaml
```

Modify the manifest file to include `runAsUser: 1001` in `securityContext`

```
kubectl create -f restrcited-pod.yaml

kubectl get pods -n restricted-ns

kubectl exec -it restricted-pod -n restricted-ns -- sh

id

kubectl delete -f restrcited-pod.yaml
```

Modes - Pod Security Admission

Create Namespace

```
apiVersion: v1
kind: Namespace
metadata:
  name: secured-ns
  labels:
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/warn: restricted
```

Testing

```
kubectl run nginx --image=nginx -n secured-ns
```

Remove the Resources Created as part of this Lab

```
kubectl delete pod nginx -n secured-ns

kubectl delete ns secured-ns
```

Points to Note - PSA and PSS

Example 1 - Mode Version

```
nano mode-version.yaml
```

```
apiVersion: v1
kind: Namespace
metadata:
```

```
name: secured-ns
labels:
  pod-security.kubernetes.io/enforce: restricted
  pod-security.kubernetes.io/enforce-version: v1.34
```

```
kubectl create -f mode-version.yaml
```

Example 2 - Workload resources and Pod templates

```
kubectl create deployment test-deployment --image=nginx --replicas=2 -n secured-ns
kubectl get deployments
kubectl get pods
```

Example 3 - Add Label to Default Namespace

```
kubectl run test-pod --image=nginx
kubectl label namespace default pod-security.kubernetes.io/enforce=restricted
```

Example 4 - Dry Run

```
kubectl label namespace default pod-security.kubernetes.io/enforce=
kubectl label --dry-run=server ns default pod-security.kubernetes.io/
enforce=restricted
```

Remove the Resources Created as part of this Lab

```
kubectl delete -f mode-version.yaml
kubectl delete pod test-pod
```

Practical - ImagePolicyWebhook

Documentation:

<https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>

Pre-Requisite

```
apt install python3-pip -y
apt install python3-venv -y
python3 -m venv myenv
source myenv/bin/activate
```

```
pip3 install flask
```

Configure Webhook

```
cd /root
```

```
nano image_policy_webhook.py
```

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Only allow nginx and httpd images
ALLOWED_IMAGES = {"nginx", "httpd"}

@app.route('/validate', methods=['POST'])
def validate():
    request_data = request.get_json()

    # Extract container images
    allowed = True
    for container in request_data.get("spec", {}).get("containers", []):
        image = container.get("image", "").split(":")[0] # Ignore tag
        if image not in ALLOWED_IMAGES:
            allowed = False
            break

    response = {
        "apiVersion": "imagepolicy.k8s.io/v1alpha1",
        "kind": "ImageReview",
        "status": {
            "allowed": allowed
        }
    }

    return jsonify(response)

if __name__ == '__main__':
    app.run(host="0.0.0.0", port=8080, debug=True)
```

```
python3 image_policy_webhook.py
```

Configure API Server (Tab-2)

```
apt install net-tools
```

```
netstat -ntlp
```

```
nano /etc/kubernetes/pki/admission-config.yaml
```

```
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
  - name: ImagePolicyWebhook
    configuration:
      imagePolicy:
        kubeConfigFile: "/etc/kubernetes/pki/webhook-kubeconfig"
```

```
allowTTL: 50
denyTTL: 50
retryBackoff: 500
defaultAllow: false
```

```
nano /etc/kubernetes/pki/webhook-kubeconfig
```

Replace the Server IP Address to your IP.

```
apiVersion: v1
kind: Config
clusters:
- cluster:
    server: http://<IP-ADDRESS>:8080/validate
    name: webhook
contexts:
- context:
    cluster: webhook
    name: webhook-context
current-context: webhook-context
```

Enable Admission Controller Plugins

```
nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

```
- --admission-control-config-file=/etc/kubernetes/pki/admission-config.yaml
```

Ensure ImagePolicyWebhook Plugin is enabled

```
- --enable-admission-plugins=NodeRestriction,ImagePolicyWebhook
```

Test the Setup

```
kubectl run nginx-pod --image=nginx
kubectl run redis-pod --image=redis
```

Practical - Kubernetes Secrets

Generating Secret Based on Generic - Literal Value

```
kubectl create secret generic firstsecret --from-literal=dbpassword=mypassword123
```

Generating Secret Based on Generic - File

```
kubectl create secret generic secondsecret --from-file=./credentials.txt
```

To View Secret from CLI

```
kubectl get secret firstsecret -o yaml
```

secret-data.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: thirdsecret
type: Opaque
data:
  username: ZGJhZG1pbG==
  password: bXlwYXNzd29yZDEyMw==
```

secret-stringdata.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: stringdata
type: Opaque
stringData:
  config.yaml: |-
    username: dbadmin
    password: mypassword
```

Practical - Configuring Cilium for Kubernetes

Configure kubeadm

```
wget https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/
sessions/cks/certified-kubernetes-security-specialist/domain-3-minimize-
microservice-vulnerability/kubeadm-master.sh

chmod +x kubeadm-master.sh

./kubeadm-master.sh
```

Install Cilium

```
CLI_ARCH=amd64

if [ "$(uname -m)" = "aarch64" ]; then CLI_ARCH=arm64; fi
curl -L --fail --remote-name-all https://github.com/cilium/cilium-cli/releases/
download/v0.16.24/cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}

sha256sum --check cilium-linux-${CLI_ARCH}.tar.gz.sha256sum

sudo tar xzvfC cilium-linux-${CLI_ARCH}.tar.gz /usr/local/bin

rm cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}
```

```
cilium install

cilium status

kubectl get nodes
```

Structure of Cilium Network Policies

Documentation Referenced:

<https://docs.cilium.io/en/latest/security/policy/index.html>

Create 2 Pods for Testing

```
kubectl run nginx --image=nginx

kubectl run curl --image=alpine/curl -- sleep 36000
```

Verify the connectivity

```
kubectl get pods -o wide

kubectl exec -it curl -- sh

curl <NGINX-IP>
```

Example 1 - Simple Deny Policy

```
nano deny.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "deny-traffic"
spec:
  endpointSelector: {}
  ingress:
    - {}
  egress:
    - {}
```

```
kubectl create -f deny.yaml

kubectl get ciliumnetworkpolicies
```

Testing - Simple Deny Policy

```
kubectl exec -it curl -- sh

curl <NGINX-IP>
```

```
ping google.com
```

```
kubectl delete -f deny.yaml
```

Example 2 - Deny Policy for Specific Pod

```
nano deny-pod.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "deny-traffic"
spec:
  endpointSelector:
    matchLabels:
      run: curl
  ingress:
    - {}
  egress:
    - {}
```

```
kubectl create -f deny-pod.yaml
```

```
kubectl get cnp
```

Testing - Policy Number 2

```
kubectl exec -it curl -- sh
ping google.com
```

```
kubectl exec -it nginx -- sh
curl google.com
```

```
kubectl delete -f deny-pod.yaml
```

Example 3 - Allow Traffic from Curl Pod to Nginx Pod

```
nano allow-curl-pod.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-curl-nginx
spec:
  endpointSelector:
    matchLabels:
      run: nginx
  ingress:
    - fromEndpoints:
```

```
- matchLabels:
  run: curl
```

```
kubectl create -f allow-curl-pod.yaml
```

Testing - Policy 3

```
kubectl exec -it curl -- sh

curl <Nginx-Pod-IP>

kubectl run random-pod --image=alpine/curl -- sleep 36000

kubectl exec -it random-pod -- sh

curl <Nginx-Pod-IP>
```

```
kubectl delete -f allow-curl-pod.yaml
```

Example 4 - Allow Egress Traffic from Curl Pod only to Nginx Pod

```
nano egress-to-nginx.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-egress-curl-to-nginx
spec:
  endpointSelector:
    matchLabels:
      run: curl
  egress:
    - toEndpoints:
        - matchLabels:
            run: nginx
```

Testing - Policy 4

```
kubectl exec -it curl -- sh

curl <Nginx-Pod-IP>

ping 8.8.8.8
```

```
kubectl delete -f egress-to-nginx.yaml
```

Delete the Resources Created in this Practical

```
kubectl delete pods --all
```

Cilium Network Policies - Entities

Create 2 Pods for Testing

```
kubectl run nginx --image=nginx

kubectl run curl --image=alpine/curl -- sleep 36000
```

Entities - Cluster

```
nano entities-cluster.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "restrict-egress-to-cluster"
spec:
  endpointSelector: {}
  egress:
    - toEntities:
      - "cluster"
```

```
kubectl create -f entities-cluster.yaml
```

Test the Setup

```
kubectl get pods -o wide

kubectl exec -it curl-pod -- sh

curl <NGINX-POD-IP>

ping google.com

curl google.com
```

```
kubectl delete -f entities-cluster.yaml
```

Entities - World

```
nano entities-world.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "restrict-egress-to-cluster"
spec:
  endpointSelector: {}
  egress:
```

```
- toEntities:
  - "world"
```

```
kubectl create -f entities-world.yaml
```

Test the Setup

```
kubectl get pods -o wide

kubectl exec -it curl-pod -- sh

curl <NGINX-POD-IP>
curl google.com
ping google.com
```

```
kubectl delete -f entities-world.yaml
```

Entities - All

```
nano entities-all.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "allow-all-egress"
spec:
  endpointSelector: {}
  egress:
    - toEntities:
      - "all"
```

```
kubectl create -f entities-all.yaml
```

Test the Setup

```
kubectl get pods -o wide

kubectl exec -it curl-pod -- sh

curl <NGINX-POD-IP>

curl google.com

ping google.com
```

```
kubectl delete -f entities-all.yaml
```

Delete the Resources Created for this Lab

```
kubectl delete pods --all
```

Cilium Network Policies - Layer 4 Rules

Documentation Referenced:

<https://docs.cilium.io/en/latest/security/policy/index.html>

Create 2 Pods for Testing

```
kubectl run nginx --image=nginx

kubectl run curl --image=alpine/curl -- sleep 36000
```

Create Cilium Network Policy

```
nano cnp-l4.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-external-80
spec:
  endpointSelector:
    matchLabels:
      run: curl
  egress:
    - toPorts:
      - ports:
        - port: "80"
          protocol: TCP
```

Test the Setup

```
kubectl get pods -o wide

kubectl exec -it curl-pod -- sh

curl <NGINX-POD-IP>

ping google.com

curl google.com
```

Delete the Resources Created for this Lab

```
kubectl delete -f cnp-l4.yaml  
  
kubectl delete pods --all
```

Cilium Network Policies - DNS Rules

Create Pod for Testing

```
kubectl run curl --image=alpine/curl -- sleep 36000
```

DNS

```
nano allow-dns.yaml
```

```
apiVersion: "cilium.io/v2"  
kind: CiliumNetworkPolicy  
metadata:  
  name: "allow-dns-example"  
spec:  
  endpointSelector: {}  
  egress:  
    - toPorts:  
      - ports:  
        - port: "53"  
        rules:  
          dns:  
            - matchName: "example.com"
```

```
kubectl create -f allow-dns.yaml
```

Testing

```
nslookup google.com  
  
nslookup example.com
```

Delete the Setup

```
kubectl delete -f allow-dns.yaml kubectl delete pod curl
```

Cilium - Deny Policies

Create 3 Pods for Testing

```
kubectl run nginx --image=nginx --labels=app=server
```

```
kubectl run random-pod --labels=app=random-pod --image=alpine/curl -- sleep 36000

kubectl run backend-pod --image=alpine/curl -- sleep 36000
```

1 - Create ingressDeny Policy

```
nano ingressDeny.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "deny-ingress"
spec:
  endpointSelector:
    matchLabels:
      app: server
  ingress:
    - fromEntities:
      - all
  ingressDeny:
    - fromEndpoints:
      - matchLabels:
          app: random-pod
```

```
kubectl create -f ingressDeny.yaml
```

Verification

```
kubectl get pods -o wide

kubectl exec -it backend-pod -- sh
curl <NGINX-POD-IP>
ping <NGINX-POD-IP>

kubectl exec -it random-pod -- sh
curl <NGINX-POD-IP>
ping <NGINX-POD-IP>
```

```
kubectl delete -f ingressDeny.yaml
```

2 - Create egressDeny Policy

```
nano egressDeny.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: "deny-egress"
spec:
  endpointSelector:
    matchLabels:
      app: random-pod
  egress:
```

```
- toEntities:
  - all
egressDeny:
- toEndpoints:
  - matchLabels:
      app: server
```

```
kubectl create -f egressDeny.yaml
```

Verification

```
kubectl get pods -o wide

kubectl exec -it random-pod -- sh
curl <NGINX-POD-IP>
ping <NGINX-POD-IP>

curl google.com
ping google.com
```

Delete the Created Resources

```
kubectl delete -f egressDeny.yaml

kubectl delete pod nginx random-pod backend-pod
```

Cilium - Transparent Encryption using IPsec

Documentation Referenced:

<https://docs.cilium.io/en/stable/security/network/encryption-ipsec/>

Install Kind

```
wget https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/
sessions/cks/certified-kubernetes-security-specialist/domain-3-minimize-
microservice-vulnerability/kind-install.sh

chmod +x kind-install.sh

./kind-install.sh
```

Configure K8s using Kind

```
nano config.yaml
```

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  image: kindest/node:v1.34.11
- role: worker
```

```
image: kindest/node:v1.34.11
- role: worker
  image: kindest/node:v1.34.11
networking:
  disableDefaultCNI: true
```

```
kind create cluster --config config.yaml
```

Install Cilium

```
CLI_ARCH=amd64

if [ "$(uname -m)" = "aarch64" ]; then CLI_ARCH=arm64; fi
curl -L --fail --remote-name-all https://github.com/cilium/cilium-cli/releases/
  download/v0.16.24/cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}

sha256sum --check cilium-linux-${CLI_ARCH}.tar.gz.sha256sum

sudo tar xzvfC cilium-linux-${CLI_ARCH}.tar.gz /usr/local/bin

rm cilium-linux-${CLI_ARCH}.tar.gz{,.sha256sum}
```

Generate and Import PSK

```
kubectl create -n kube-system secret generic cilium-ipsec-keys \
  --from-literal=keys="3+ rfc4106(gcm(aes)) $(echo $(dd if=/dev/urandom count=20
    bs=1 2> /dev/null | xxd -p -c 64)) 128"

kubectl -n kube-system get secrets cilium-ipsec-keys
```

Enable Transparent Encryption in Cilium (IPSec)

```
cilium install --version 1.17.1 --set encryption.enabled=true --set
  encryption.type=ipsec

cilium status

cilium config view | grep enable-ipsec

kubectl get nodes
```

Testing the Setup

Launch 2 Pods in different worker node (Terminal Tab 1)

```
nano curl-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: curl
spec:
  nodeSelector:
    kubernetes.io/hostname: kind-worker
```

```
containers:
- name: busybox
  image: alpine/curl
  command: ["sleep", "36000"]
```

```
kubectl create -f curl-pod.yaml
```

```
nano nginx-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeSelector:
    kubernetes.io/hostname: kind-worker2
  containers:
  - name: nginx
    image: nginx
```

```
kubectl create -f nginx-pod.yaml
```

Run a bash shell in one of the Cilium pods (Terminal Tab 2)

```
kubectl -n kube-system exec -ti ds/cilium -- bash
```

Install tcpdump and check if traffic is encrypted

```
apt-get update
apt-get -y install tcpdump
```

```
tcpdump -n -i cilium_vxlan esp
```

In Terminal Tab 1

```
kubectl get pods -o wide

kubectl exec -it curl -- sh

curl <NGINX-POD-IP>
```

Delete the Kind Cluster

```
kind delete cluster
```

Cilium - Transparent Encryption using WireGuard

Documentation Referenced:

<https://docs.cilium.io/en/stable/security/network/encryption-wireguard/>

Install Kind

```
wget https://raw.githubusercontent.com/mevijays/training-k8s/refs/heads/main/
sessions/cks/certified-kubernetes-security-specialist/domain-3-minimize-
microservice-vulnerability/kind-install.sh

chmod +x kind-install.sh

./kind-install.sh
```

Configure K8s using Kind

```
nano config.yaml
```

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  image: kindest/node:v1.34.11
- role: worker
  image: kindest/node:v1.34.11
- role: worker
  image: kindest/node:v1.34.11
networking:
  disableDefaultCNI: true
```

```
kind create cluster --config config.yaml
```

Enable Transparent Encryption in Cilium (WireGuard)

```
cilium install --version 1.17.1 --set encryption.enabled=true --set
encryption.type=wireguard

cilium status

cilium config view | grep enable-wireguard
```

Testing the Setup

Launch 2 Pods in different worker node (Terminal Tab 1)

```
nano curl-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
```

```

name: curl
spec:
  nodeSelector:
    kubernetes.io/hostname: kind-worker
  containers:
  - name: busybox
    image: alpine/curl
    command: ["sleep", "36000"]

```

```
kubectl create -f curl-pod.yaml
```

```
nano nginx-pod.yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  nodeSelector:
    kubernetes.io/hostname: kind-worker2
  containers:
  - name: nginx
    image: nginx

```

```
kubectl create -f nginx-pod.yaml
```

Run a bash shell in one of the Cilium pods (Terminal Tab 2)

```
kubectl -n kube-system exec -ti ds/cilium -- bash
```

Install tcpdump and check if traffic is encrypted

```

apt-get update
apt-get -y install tcpdump

```

```
tcpdump -n -i cilium_wg0 -nn -vv
```

In Terminal Tab 1

```

kubectl get pods -o wide

kubectl exec -it curl-pod -- sh

curl <NGINX-POD-IP>

```

Security Context - readOnlyRootFilesystem

Documentation Referred:

<https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>

<https://kubernetes.io/docs/concepts/storage/volumes/#emptydir>

Security Context of readOnlyRootFilesystem

```
nano security-context-ro.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: readonly-pod
spec:
  containers:
  - name: demo
    image: busybox:1.37
    command: [ "sleep", "1h" ]
    securityContext:
      readOnlyRootFilesystem: true
```

```
kubectl create -f security-context-ro.yaml
```

Testing

```
kubectl exec -it readonly-pod -- sh

touch test.txt

cd tmp

touch test.txt
```

Security Context of readOnlyRootFilesystem + emptyDir

```
nano security-context-ro-empty-dir.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: readonly-pod-emptydir
spec:
  containers:
  - name: my-container
    image: busybox:1.37
    command: [ "sleep", "1h" ]
    securityContext:
      readOnlyRootFilesystem: true
  volumeMounts:
  - name: tmp-storage
```

```

    mountPath: /tmp
  volumes:
    - name: tmp-storage
      emptyDir: {}

```

```
kubectl create -f security-context-ro-empty-dir.yaml
```

Testing

```

kubectl exec -it readonly-pod-emptydir -- sh

touch test.txt

cd tmp

touch test.txt

```

Security Context of readOnlyRootFilesystem - Nginx Example

```
nano security-context-ro-empty-dir.yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-ro
spec:
  containers:
    - name: my-container
      image: nginx
      securityContext:
        readOnlyRootFilesystem: true

```

```

kubectl create -f security-context-ro-empty-dir.yaml

kubectl logs nginx-ro

```

Practical - Installing Istio

```

curl -L https://istio.io/downloadIstio | sh -

cd istio-*

export PATH=$PWD/bin:$PATH

istioctl version

istioctl install --set profile=demo -y

kubectl get pods -n istio-system

```

Istio - Automatic Sidecar Proxy Injection

Verify Proxy Injection Before Setting Label

```
kubectl run nginx-01 --image=nginx  
  
kubectl get pods
```

Set Label for Automatic Sidecar Injection

```
kubectl get namespace default --show-labels  
  
kubectl label namespace default istio-injection=enabled  
  
kubectl get namespace default --show-labels  
  
kubectl run nginx-02 --image=nginx
```

Verification

```
kubectl get pods  
  
kubectl describe pod nginx-02
```

Practical - Mutual TLS with Istio

1 - Remove Side car injection label from namespace

```
kubectl label namespace default istio-injection-
```

2 - Create Two Pods without sidecar proxy

```
kubectl run nginx-01 --image=nginx  
  
kubectl run curl-01 --image=alpine/curl -- sleep 36000
```

3- Enable Side car injection for namespace

```
kubectl label namespace default istio-injection=enabled
```

4 - Create Two pods with side car proxy

```
kubectl run nginx-02 --image=nginx  
  
kubectl run curl-02 --image=alpine/curl -- sleep 36000
```

5 - Create necessary service for Nginx Pods

```
kubectl label pods nginx-01 app=nginx-01

kubectl label pods nginx-02 app=nginx-02

kubectl expose pod nginx-01 --port=80 --name=nginx-01-service

kubectl expose pod nginx-02 --port=80 --name=nginx-02-service
```

6 - Enable Peer Authentication

```
nano peer-auth.yaml
```

```
apiVersion: security.istio.io/v1
kind: PeerAuthentication
metadata:
  name: default
spec:
  mtls:
    mode: STRICT
```

```
kubectl apply -f peer-auth.yaml
```

```
kubectl get peerauthentication
```

7 - Testing

```
kubectl exec -it curl-01 -- sh

curl http://nginx-01-service.default.svc.cluster.local

curl http://nginx-02-service.default.svc.cluster.local
```

```
kubectl exec -it curl-02 -- sh

curl http://nginx-01-service.default.svc.cluster.local

curl http://nginx-02-service.default.svc.cluster.local
```

8 - Check Certificate Related Information

```
istioctl proxy-config secret curl-01

istioctl proxy-config secret curl-02

istioctl proxy-config secret nginx-02
```

Reference - HostPath Volumes

Create a HostPath Volume:

```
nano host-path.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
    - image: nginx
      name: test-container
      volumeMounts:
        - mountPath: /host-path
          name: host-volume
  volumes:
    - name: host-volume
      hostPath:
        path: /
```

```
kubectl apply -f host-path.yaml
kubectl exec -it hostpath-pod -- bash
```

Improved Pod Security Policy:

```
nano /root/psp/restrictive-psp.yaml
```

```
apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: restrictive-psp
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfileNames: '*'
spec:
  privileged: false
  allowPrivilegeEscalation: true
  allowedCapabilities:
    - '*'
  volumes:
    - configMap
    - downwardAPI
    - emptyDir
    - persistentVolumeClaim
    - secret
    - projected
  hostNetwork: true
  hostPorts:
    - min: 0
      max: 65535
  hostIPC: true
  hostPID: true
  runAsUser:
    rule: 'RunAsAny'
  selinux:
```

```

    rule: 'RunAsAny'
  supplementalGroups:
    rule: 'RunAsAny'
  fsGroup:
    rule: 'RunAsAny'

```

Reference - Host PID Namespace

Sample YAML File based on Host PID:

```
nano host-pid.yaml
```

```

apiVersion: v1
kind: Pod
metadata:
  name: hostpid
spec:
  hostPID: true
  containers:
  - image: busybox
    name: demo-pod
    command:
    - sleep
    - "3600"

```

```
kubectl apply -f host-pid.yaml
```

Create Sample Pods:

```

kubectl run httpd-pod-1 --image=httpd
kubectl run httpd-pod-2 --image=httpd

```

Connect to hostpid POD:

```

kubectl exec -it hostpid -- sh
ps -ef | grep httpd
kill -9 PID && kill -9 PID

```

Improved Pod Security Policy:

```

apiVersion: policy/v1beta1
kind: PodSecurityPolicy
metadata:
  name: restrictive-psp
  annotations:
    seccomp.security.alpha.kubernetes.io/allowedProfileNames: '*'
spec:
  privileged: false
  allowPrivilegeEscalation: true
  allowedCapabilities:
  - '*'
  volumes:

```

```

- configMap
- downwardAPI
- emptyDir
- persistentVolumeClaim
- secret
- projected
hostNetwork: true
hostPorts:
- min: 0
  max: 65535
hostIPC: true
hostPID: false
runAsUser:
  rule: 'RunAsAny'
seLinux:
  rule: 'RunAsAny'
supplementalGroups:
  rule: 'RunAsAny'
fsGroup:
  rule: 'RunAsAny'

```

Reapply the restrictive policy to not allow hostpid:

```
kubectl apply -f restrictive-psp.yaml
```

Reference - Mounting Secrets into Pods

secretpod.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: secretmount
spec:
  containers:
  - name: secretmount
    image: nginx
    volumeMounts:
    - name: foo
      mountPath: "/etc/foo"
      readOnly: true
  volumes:
  - name: foo
    secret:
      secretName: firstsecret

```

Mounting Secrets as Environment Variables

File Name: secret-env.yaml

```

apiVersion: v1
kind: Pod
metadata:
  name: secret-env
spec:
  containers:

```

```
- name: secret-env
  image: nginx
  env:
    - name: SECRET_USERNAME
      valueFrom:
        secretKeyRef:
          name: firstsecret
          key: dbpass
  restartPolicy: Never
```

Reference - Custom ImagePolicyWebhook Configuration

GitHub Page:

<https://github.com/flavio/kube-image-bouncer>

Switch to the controlconf directory:

```
cd /etc/kubernetes/controlconf
```

Creating Custom Webhook Configuration File:

```
nano custom-config.yaml
```

```
apiVersion: apiserver.config.k8s.io/v1
kind: AdmissionConfiguration
plugins:
- name: ImagePolicyWebhook
  configuration:
    imagePolicy:
      kubeConfigFile: /etc/kubernetes/controlconf/custom-webhook.kubeconfig
      allowTTL: 50
      denyTTL: 50
      retryBackoff: 500
      defaultAllow: false
```

Creating Custom Webhook kubeconfigfile:

```
nano custom-webhook.kubeconfig
```

```
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority: /etc/kubernetes/controlconf/webhook.crt
    server: https://bouncer.local.lan:1323/image_policy
    name: bouncer_webhook
contexts:
- context:
    cluster: bouncer_webhook
    user: api-server
    name: bouncer_validator
```

```
current-context: bouncer_validator
preferences: {}
users:
- name: api-server
  user:
    client-certificate: /etc/kubernetes/controlconf/api-user.crt
    client-key: /etc/kubernetes/controlconf/api-user.key
```

Modify the kube-apiserver Manifest:

```
nano /etc/kubernetes/manifests/kube-apiserver.yaml
```

Setting Host Alias:

```
hostAliases:
- ip: "<IP-ADDRESS>"
  hostnames:
  - "bouncer.local.lan"

- mountPath: /etc/kubernetes/controlconf
  name: admission-controller
  readOnly: true
```

```
- hostPath:
  path: /etc/kubernetes/controlconf
  type: DirectoryOrCreate
  name: admission-controller
```

```
- --admission-control-config-file=/etc/kubernetes/controlconf/custom-config.yaml
```

Start the webhook service in a different tab:

```
kube-image-bouncer --cert webhook.crt --key webhook.key
```

Verification:

```
kubectrl run dem-nginx --image=nginx:latest
kubectrl run dem-nginx --image=nginx:1.19.2
```

Reference - Cilium Network Policy for Services

Create Base Pods and Service

```
kubectrl run nginx-1 --image=nginx

kubectrl expose pod nginx-1 --port=80 --name=nginx-service --target-port=80

kubectrl run curl-pod --image=alpine/curl --command -- sleep infinity
```

Create CNP

```
nano cnp-service.yaml
```

```
apiVersion: "cilium.io/v2"
kind: CiliumNetworkPolicy
metadata:
  name: allow-curl-to-nginx-service
spec:
  endpointSelector:
    matchLabels:
      run: curl-pod
  egress:
    - toServices:
      - k8sService:
          serviceName: nginx-service
          namespace: default
        toPorts:
          - ports:
              - port: "80"
              protocol: TCP
```

```
kubectl create -f cnp-service.yaml
```

Testing

```
kubectl exec curl-pod -- curl <NGINX-SERVICE-IP>
```

Reference - Attack Case Study - Reading a Protected Pod

Step 1 - Create a Non-Privileged Super Secret Pod:

```
nano super-secret-pod.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: super-secret
  namespace: development
spec:
  containers:
    - image: nginx
      name: demo-pod
```

```
kubectl apply -f super-secret-pod.yaml
```

Step 2 - Create a Privileged Pod:

```
nano privileged.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: privileged
  namespace: development
spec:
  containers:
  - image: nginx
    name: demo-pod
    securityContext:
      privileged: true
```

```
kubectl apply -f privileged.yaml
```

Step 3 - Store Secret Data in Super Secret Pod:

```
kubectl exec -n development -it super-secret -- bash
echo "TOP Secret" > /root/super-secret.txt
```

logout of the super-secret pod

Step 4 - Connect to Privileged Pod:

Note: The disk name might be different in your-case. Please ensure you use the right naming while mounting.

```
kubectl exec -n development -it privileged -- bash
mkdir /host-data
```

```
mount /dev/vda1 /host-data
cd /host-data
cd /host-data/var/lib/docker/overlay2
find . -name super-secret.txt
```

Domain 4 · System Hardening

Reducing the attack surface of the node: AppArmor profiles, the container runtime layer (containerd, runc, CRI), sandboxed runtimes with gVisor and host-level network policy.

Practical - AppArmor

Check status of apparmor:

```
systemctl status apparmor  
  
aa-status
```

Sample Script:

```
mkdir /root/apparmor  
  
cd /root/apparmor
```

```
nano app.sh
```

```
#!/bin/bash  
touch /tmp/file.txt  
echo "New File created"  
  
rm -f /tmp/file.txt  
echo "New file removed"
```

```
chmod +x app.sh
```

Install Apparmor Utils:

```
apt install apparmor-utils -y
```

Generate a new profile:

```
aa-genprof ./app.sh
```

```
./app.sh (from new tab)
```

Verify the new profile:

```
cat /etc/apparmor.d/root.apparmor.app.sh
aa-status
```

Disable a profile:

```
ln -s /etc/apparmor.d/root.apparmor.app.sh /etc/apparmor.d/disable/
apparmor_parser -R /etc/apparmor.d/root.apparmor.app.sh
```

AppArmor and Kubernetes

Create a Sample Profile:

```
apparmor_parser -q <<EOF
#include <tunables/global>

profile k8s-apparmor-example-deny-write flags=(attach_disconnected) {
  #include <abstractions/base>

  file,

  # Deny all file writes.
  deny /** w,
}
EOF
```

Verify the status of the profile:

```
aa-status
```

Sample YAML File based on Host PID:

```
cd /root
```

```
nano hello-armor.yaml
```

```
apiVersion: v1
kind: Pod
metadata:
  name: hello-apparmor
spec:
  securityContext:
    appArmorProfile:
      type: Localhost
      localhostProfile: k8s-apparmor-example-deny-write
  containers:
    - name: hello
```

```
image: busybox
command: [ "sh", "-c", "echo 'Hello AppArmor!' && sleep 1h" ]
```

```
kubectl apply -f hello-armor.yaml
```

Verification Stage:

```
kubectl exec -it hello-apparmor -- sh
touch /tmp/file.txt
```

Configuring containerd and runc

Documentation:

<https://kubernetes.io/docs/setup/production-environment/container-runtimes/>

<https://www.cncf.io/projects/>

Install containerd:

```
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

```
modprobe overlay
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
```

```
sysctl --system
```

```
apt-get install -y containerd
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml
systemctl restart containerd
```

Create Container with Containerd:

```
ctr image pull docker.io/library/nginx:latest
ctr image ls
ctr container create docker.io/library/nginx:latest nginx
ctr container list
```

Get the snapshot to get the contents:

```
mkdir /root/nginx-rootfs
ctr snapshot mounts nginx-rootfs/ nginx | bash
```

Generate the config.yaml:

```
cd /root
runc spec
```

Modify the config.yaml to include the name of nginx-rootfs

Create a container from runc:

```
runc run mycontainer
```

Container Runtime Interface

Pre-Requisite:

containerd installed and running.

Step 1 Configuring Repo and Installation:

```
curl -s https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -
```

```
cat <<EOF | sudo tee /etc/apt/sources.list.d/kubernetes.list
deb https://apt.kubernetes.io/ kubernetes-xenial main
EOF
```

```
apt-get update
apt-get install -y kubelet=1.20.1-00 kubeadm=1.20.1-00 kubectl=1.20.1-00
```

Step 2 - Initialize:

```
kubeadm init --pod-network-cidr=10.244.0.0/16
```

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 3 Install Network Addon (flannel):

```
kubectl apply -f https://raw.githubusercontent.com/coreos/flannel/master/Documentation/kube-flannel.yml
```

Step 4 Remove the Taint:

```
kubectl taint nodes --all node-role.kubernetes.io/master-
```

Implementing RuntimeClass - gVisor**Documentation:**

<https://kubernetes.io/docs/concepts/containers/runtime-class/>

Step 1 - Configure Docker:

```
# Add Docker's official GPG key:
sudo apt update && apt -y install ca-certificates curl

sudo install -m 0755 -d /etc/apt/keyrings

sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
    docker.asc

sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

```
sudo apt-get -y install docker-ce docker-ce-cli containerd.io
```

Step 2 - Install Configure Minikube:

```
wget https://github.com/kubernetes/minikube/releases/download/v1.37.0/minikube-linux-
    amd64

mv minikube-linux-amd64 minikube

chmod +x minikube

sudo mv ./minikube /usr/local/bin/minikube
```

```
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/
    stable.txt)/bin/linux/amd64/kubectl"

sudo mv kubectl /usr/local/bin

chmod +x /usr/local/bin/kubectl
```

Step 3 - Create a New user for testing

```

useradd -m -s /bin/bash -G sudo,docker devuser
visudo -f /etc/sudoers.d/devuser
devuser ALL=(ALL) NOPASSWD:ALL
su - devuser

```

Step 4: Start Minikube:

```

minikube start --container-runtime=containerd --docker-opt containerd=/var/run/
containerd/containerd.sock

```

Step 5 Enable gVisor addon:

```

minikube addons list
minikube addons enable gvisor
minikube addons list

```

Step 6 - Explore gVisor:

```

kubectl get runtimeclass

```

```

nano runtimeclass.yaml

```

```

apiVersion: node.k8s.io/v1
kind: RuntimeClass
metadata:
  name: gvisor2
handler: runsc

```

```

kubectl apply -f runtimeclass.yaml

```

```

nano gvisor-pod.yaml

```

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  runtimeClassName: gvisor2
  containers:
  - image: nginx
    name: nginx

```

```

kubectl apply -f gvisor-pod.yaml

```

Create one more pod for seeing the difference in dmesg and uname output:

```
kubectl run nginx-default --image=nginx
```

Verify output (dmesg and uname -r)

From host machine: Run `dmesg` and `'uname -r'`

```
kubectl exec -it nginx -- bash
dmesg
uname -r
logout
```

```
kubectl exec -it nginx-default -- bash
dmesg
uname -r
logout
```

Reference - Network Policies Practical

Create base setup:

```
kubectl create ns external
```

```
kubectl run pod-1 --image=praqma/network-multitool
kubectl run pod-2 --image=praqma/network-multitool
kubectl run pod-3 --image=praqma/network-multitool -n external
```

```
kubectl get pods -o wide
kubectl get pods -o wide -n external
```

Check connectivity from POD-1 to POD2 and POD-3 and Internet:

```
kubectl exec -it pod-1 -- ping [pod-2-ip]
kubectl exec -it pod-1 -- ping [pod-3-ip]
kubectl exec -it pod-1 -- ping google.com
```

Rule 1: Deny Ingress

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
```

```
policyTypes:
```

```
- Ingress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl get netpol
```

```
kubectl exec -it pod-1 -- ping [pod2-ip]
kubectl exec -it pod-1 -- ping [pod3-ip]
kubectl exec -it pod-1 -- ping google.com

kubectl exec -n external -it pod-3 -- ping [pod1-ip]
kubectl exec -n external -it pod-3 -- ping [pod2-ip]
```

```
kubectl delete -f netpol.yaml
```

[Remove the contents of netpol.yaml file]

Rule 2: Allow Ingress

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl exec -n external -it pod-3 -- ping [pod1-ip]
kubectl exec -n external -it pod-3 -- ping [pod2-ip]
```

```
kubectl delete -f netpol.yaml
```

[Remove the contents of netpol.yaml file]

Rule 3: Deny All Egress:

```
nano netpol.yaml
```

```

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
spec:
  podSelector: {}
  policyTypes:
  - Egress

```

```
kubectl apply -f netpol.yaml
```

```

kubectl exec -it pod-2 -- ping google.com
kubectl exec -n external -it pod-3 -- ping [pod1]
kubectl exec -n external -it pod-3 -- ping [pod2]

```

```
kubectl delete -f netpol.yaml
```

Rule 4 - PodSelector:

```
nano netpol.yaml
```

```

---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: podselector-suspicious
spec:
  podSelector:
    matchLabels:
      role: suspicious
  policyTypes:
  - Ingress
  - Egress

```

```
kubectl apply -f netpol.yaml
```

```

kubectl label pod pod-1 role=suspicious
kubectl get pods --show-labels
kubectl exec -it pod-2 -- ping [pod-1]
kubectl exec -it pod-1 -- ping google.com

```

Remove the label and verify:

```

kubectl label pod pod-1 role-
kubectl exec -it pod-1 -- ping google.com

```

```
kubectl delete -f netpol.yaml
```

Reference - Network Policies Practical (Variant)

Create base setup:

```
kubectl create ns external
```

```
kubectl run pod-1 --image=pragma/network-multitool
kubectl run pod-2 --image=pragma/network-multitool
kubectl run pod-3 --image=pragma/network-multitool -n external
```

```
kubectl get pods -o wide
kubectl get pods -o wide -n external
```

Check connectivity from POD-1 to POD2 and POD-3 and Internet:

```
kubectl exec -it pod-1 -- ping [pod-2-ip]
kubectl exec -it pod-1 -- ping [pod-3-ip]
kubectl exec -it pod-1 -- ping google.com
```

Rule 1: Deny Ingress

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl get netpol
```

```
kubectl exec -it pod-1 -- ping [pod2-ip]
kubectl exec -it pod-1 -- ping [pod3-ip]
kubectl exec -it pod-1 -- ping google.com

kubectl exec -it pod-3 -- ping [pod1-ip]
kubectl exec -it pod-3 -- ping [pod2-ip]
```

```
kubectl delete -f netpol.yaml
```

[Remove the contents of netpol.yaml file]

Rule 2: Allow Ingress

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}
  ingress:
  - {}
  policyTypes:
  - Ingress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl exec -it pod-3 -- ping [pod1-ip]
kubectl exec -it pod-3 -- ping [pod2-ip]
```

```
kubectl delete -f netpol.yaml
```

[Remove the contents of netpol.yaml file]

Rule 3: Deny All Egress:

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
spec:
  podSelector: {}
  policyTypes:
  - Egress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl exec -it pod-2 -- ping google.com
kubectl exec -n external -it pod-3 -- ping [pod1]
kubectl exec -n external -it pod-3 -- ping [pod2]
```

```
kubectl delete -f netpol.yaml
```

Rule 4 - PodSelector:

```
nano netpol.yaml
```

```
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: podselector-suspicious
spec:
  podSelector:
    matchLabels:
      role: suspicious
  policyTypes:
  - Ingress
  - Egress
```

```
kubectl apply -f netpol.yaml
```

```
kubectl label pod pod-1 role=suspicious
kubectl get pods --show-labels
kubectl exec -it pod-2 -- ping [pod-1]
kubectl exec -it pod-1 -- ping google.com
```

Remove the label and verify:

```
kubectl label pod pod-1 role-
kubectl exec -it pod-1 -- ping google.com
```

```
kubectl delete -f netpol.yaml
```

Reference - kubeadm Cluster with Calico

Step 1: Setup containerd

```
cat <<EOF | sudo tee /etc/modules-load.d/containerd.conf
overlay
br_netfilter
EOF
```

```
modprobe overlay
modprobe br_netfilter
```

```
cat <<EOF | sudo tee /etc/sysctl.d/99-kubernetes-cri.conf
net.bridge.bridge-nf-call-iptables = 1
net.ipv4.ip_forward = 1
net.bridge.bridge-nf-call-ip6tables = 1
EOF
```

```
sysctl --system
```

```
apt-get install -y containerd
mkdir -p /etc/containerd
containerd config default > /etc/containerd/config.toml
```

```
nano /etc/containerd/config.toml
```

--> SystemdCgroup = true

```
systemctl restart containerd
```

Step 2: Kernel Parameter Configuration

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-ip6tables = 1
net.bridge.bridge-nf-call-iptables = 1
EOF
```

```
sudo sysctl --system
```

Step 3: Configuring Repo and Installation

```
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
sudo curl -fsSL https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo gpg --
dearmor -o /etc/apt/keyrings/kubernetes-archive-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-archive-keyring.gpg] https://
apt.kubernetes.io/ kubernetes-xenial main" | sudo tee /etc/apt/
sources.list.d/kubernetes.list
```

```
sudo apt-get update
apt-cache madison kubeadm
sudo apt-get install -y kubelet=1.24.2-00 kubeadm=1.24.2-00 kubectl=1.24.2-00 cri-
tools=1.24.2-00
sudo apt-mark hold kubelet kubeadm kubectl
```

Step 4 - Initialize Cluster with kubeadm:

```
kubeadm init --pod-network-cidr=192.168.0.0/16 --kubernetes-version=1.24.2
```

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 5 - Remove the Taint:

```
kubectl taint nodes --all node-role.kubernetes.io/master-  
kubectl taint nodes --all node-role.kubernetes.io/control-plane-
```

Step 6 - Install Network Addon (Calico):

```
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.4/  
manifests/tigera-operator.yaml  
kubectl create -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.4/  
manifests/custom-resources.yaml
```

7 - Verification:

```
kubectl get nodes  
kubectl run nginx --image=nginx  
kubectl get pods
```

Domain 5 · Supply Chain Security

Trusting what you run: image vulnerability scanning, cluster benchmarking, static manifest analysis, Dockerfile best practices, securing the Docker daemon and generating an SBOM.

Scan Images for Known Vulnerabilities

Official Repository:

<https://github.com/aquasecurity/trivy>

Installing Trivy:

```
curl -sfl https://raw.githubusercontent.com/aquasecurity/trivy/master/contrib/install.sh | sh -s -- -b /usr/local/bin
```

DockerHub page for Nginx:

```
https://hub.docker.com/\_/nginx
```

Trivy Command to Scan an Image:

```
trivy image nginx:1.19.5
```

Scanning K8s Clusters for Security Best Practices

Documentation:

<https://github.com/aquasecurity/kube-bench#download-and-install-binaries>

Installation Steps:

```
curl -L https://github.com/aquasecurity/kube-bench/releases/download/v0.3.1/kube-bench_0.3.1_linux_amd64.deb -o kube-bench_0.3.1_linux_amd64.deb
```

```
sudo apt install ./kube-bench_0.3.1_linux_amd64.deb -f
```

Running kube-bench:

```
kube-bench
```

Static Analysis

Documentation:

<https://github.com/bridgecrewio/checkov>

Install the Tool:

```
apt install python3-pip  
pip3 install checkov
```

Our Demo Manifest File:

```
nano pod-priv.yaml
```

```
apiVersion: v1  
kind: Pod  
metadata:  
  name: privileged  
spec:  
  containers:  
  - image: nginx  
    name: demo-pod  
    securityContext:  
      privileged: true
```

Perform a static analysis:

```
checkov -f pod-priv.yaml
```

Dockerfile - Security Best Practices

```
nano app.sh
```

```
#!/bin/sh  
echo "Hello, Docker"
```

Base Dockerfile

```
nano Dockerfile
```

```
FROM ubuntu:18.04
USER root
RUN apt-get update
RUN apt-get install -y curl
RUN apt-get install -y wget
RUN apt-get install -y nano
EXPOSE 80
WORKDIR /usr/src/app
COPY app.sh .
RUN chmod -R 777 /usr/src/app/app.sh
CMD ["/app.sh"]
```

Verification

```
docker build -t myapp:v1 .

docker images

docker run myapp:v1
```

Revision 1 - DockerFile

```
echo > Dockerfile
```

```
FROM ubuntu:25.10

RUN apt-get update && \
    apt-get install -y curl && \
    apt-get install -y wget && \
    apt-get install -y nano && \
    useradd appuser

EXPOSE 80
WORKDIR /usr/src/app
COPY app.sh .

RUN chmod 700 app.sh && chown appuser:appuser app.sh
USER appuser
CMD ["/app.sh"]
```

Verification

```
docker build -t myapp:v2 .

docker images

docker run myapp:v2
```

Revision 2 DockerFile

```
echo > Dockerfile
```

```
FROM alpine:latest
WORKDIR /usr/src/app
COPY app.sh .
RUN chmod +x app.sh
RUN adduser -D appuser && chown appuser:appuser app.sh
USER appuser
CMD ["/usr/src/app/app.sh"]
```

Verification

```
docker build -t myapp:v3 .

docker images

docker run myapp:v3
```

Securing the Docker Daemon

Install Docker in Ubuntu

```
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/
  docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
  docker-compose-plugin
```

```
systemctl status docker
```

Docker Group Security

```
useradd test-user

id test-user

usermod -aG docker test-user
passwd test-user

su - test-user

sudo su -

docker run --rm -it --privileged --net=host -v /:/mnt ubuntu chroot /mnt bash
```

```
whoami
```

```
useradd attacker
```

```
usermod -aG root attacker
```

Logout from container

```
userdel attacker
```

```
gpasswd -d test-user docker
```

```
gpasswd -d attacker root
```

Expose the Docker API

```
nano /usr/lib/systemd/system/docker.service
```

```
ExecStart=/usr/bin/dockerd -H unix:///var/run/docker.sock -H tcp://127.0.0.1:2375 --  
containerd=/run/containerd/containerd.sock
```

```
systemctl daemon-reload
```

```
systemctl restart docker
```

Testing if Docker API Is Exposed

An attacker can check if the Docker remote API is accessible using curl:

```
curl http://127.0.0.1:2375/version
```

An attacker can list all running containers:

```
docker run -d nginx
```

```
curl http://127.0.0.1:2375/containers/json
```

Create New Container

```
curl -X POST -H "Content-Type: application/json" --data '{"Image": "nginx", "Cmd":  
["tail", "-f", "/dev/null"], "HostConfig": {"Privileged": true}}' http://  
127.0.0.1:2375/containers/create
```

Start the Container

```
curl -X POST http://127.0.0.1:2375/containers/<CONTAINER_ID>/start
```

Reverting the Changes:

1. Remove Docker API from `/usr/lib/systemd/system/docker.service`

```
systemctl daemon-reload

systemctl stop docker

systemctl stop docker.socket

systemctl start docker

docker ps -a
docker rm <container-name>

docker images
docker rmi <image-names>
```

Practical - Docker Daemon Socket over TLS

Generate CA Key and Certificate

```
mkdir -p /etc/docker/certs
cd /etc/docker/certs

openssl genpkey -algorithm RSA -out ca-key.pem

openssl req -new -x509 -days 365 -key ca-key.pem -subj "/CN=MyDockerCA" -out ca.pem
```

Generate Server Certificate and Key

```
openssl genpkey -algorithm RSA -out server-key.pem

openssl req -new -key server-key.pem -subj "/CN=demo.docker.internal" -out server.csr

openssl x509 -req -in server.csr -CA ca.pem -CAkey ca-key.pem -CAcreateserial -days 365 -out server-cert.pem
```

Generate Client Certificate and Key

```
openssl genpkey -algorithm RSA -out client-key.pem

openssl req -new -key client-key.pem -subj "/CN=client" -out client.csr

openssl x509 -req -in client.csr -CA ca.pem -CAkey ca-key.pem -CAcreateserial -days 365 -out client-cert.pem
```

Configure Docker Daemon

```
nano /etc/docker/daemon.json
```

```
{
  "tls": true,
```

```
"tlsverify": true,
"tlscacert": "/etc/docker/certs/ca.pem",
"tlscert": "/etc/docker/certs/server-cert.pem",
"tlskey": "/etc/docker/certs/server-key.pem",
"hosts": ["tcp://0.0.0.0:2376", "unix:///var/run/docker.sock"]
}
```

Modify systemd service file

```
nano /usr/lib/systemd/system/docker.service
```

Modify the `ExecStart` to only use dockerd without any flags

```
ExecStart=/usr/bin/dockerd
```

Restart Docker

```
systemctl daemon-reload
systemctl restart docker
```

Testing the Setup

```
apt-get install net-tools
```

```
netstat -ntlp
```

```
curl http://127.0.0.1:2376/version
```

```
curl --cert /etc/docker/certs/client-cert.pem --key /etc/docker/certs/client-key.pem
--cacert /etc/docker/certs/ca.pem https://127.0.0.1:2376/version
```

```
nano /etc/hosts
```

Add following line

```
127.0.0.1 demo.docker.internal
```

```
curl --cert /etc/docker/certs/client-cert.pem --key /etc/docker/certs/client-key.pem
--cacert /etc/docker/certs/ca.pem https://demo.docker.internal:2376/version
```

SBOM Practical

Pages Referred:

<https://github.com/kubernetes-sigs/bom>

<https://trivy.dev/v0.33/docs/sbom/>

Install Bom

```
wget https://github.com/kubernetes-sigs/bom/releases/download/v0.6.0/bom-amd64-linux

mv bom-amd64-linux bom

chmod +x bom

mv bom /usr/local/bin
```

Install Trivy

```
curl -sL https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/
install.sh | sh

sudo mv bin/trivy /usr/local/bin
```

Bom Learnings

```
bom generate spdx-json --image nginx:latest --output nginx.spdx.json

bom generate spdx-json --image
nginx@sha256:28edb1806e63847a8d6f77a7c312045e1bd91d5e3c944c8a0012f0b14c830c44
--output nginx.spdx.json

bom document outline nginx.spdx.json

bom document outline nginx-spdx.json | grep dpkg
```

Trivy Learnings

```
trivy image --format spdx-json --output nginx-spdx.json nginx:latest

trivy image --format cyclonedx --output nginx-cyclone.json nginx:latest

trivy sbom nginx-spdx.json
```

Reference - Running the Docker Daemon Manually

Run Docker through CLI

```
systemctl stop docker
systemctl stop docker.socket

dockerd
```

Test Container Launch

```
docker run -d --name nginx nginx

docker exec -it nginx bash
```

```
cat /etc/resolv.conf
```

Run Docker through CLI with DNS Flag

```
dockerd --dns=8.8.8.8  
  
docker run -d --name nginx2 nginx  
  
docker exec -it nginx2 bash  
  
cat /etc/resolv.conf
```

Configure daemon.json file

```
nano /etc/docker/daemon.json
```

```
{  
  "dns": ["8.8.4.4"]  
}
```

```
dockerd  
  
docker run -d --name nginx3 nginx  
  
docker exec -it nginx3 bash  
  
cat /etc/resolv.conf
```

Domain 6 · Monitoring, Logging and Runtime Security

Detecting what happens at runtime: installing and configuring Falco, writing custom rules and macros, Sysdig basics and detailed API server audit logging.

Installing Falco

Falco Documentation:

<https://falco.org/docs/getting-started/installation/>

Installation Steps:

```
curl -s https://falco.org/repo/falcosecurity-3672BA8F.asc | apt-key add -  
echo "deb https://dl.bintray.com/falcosecurity/deb stable main" | tee -a /etc/apt/  
sources.list.d/falcosecurity.list  
apt-get -y install linux-headers-$(uname -r)  
apt-get update && apt-get install -y falco
```

Start falco:

```
falco
```

Sample Rules tested:

```
kubectl run nginx --image=nginx  
kubectl exec -it nginx -- bash
```

```
mkdir /bin/tmp-dir  
cat /etc/shadow
```

Falco - Practical

Monitor Falco Logs (Terminal Tab 1)

```
systemctl status falco  
  
journalctl -u falco-modern-bpf.service -f
```

Testing

```
cat /etc/shadow

kubectl run nginx-pod --image=nginx

kubectl exec -it nginx-pod -- bash
```

Writing Custom Falco Rules and Macros

Documentation Refernced:

<https://falco.org/docs/concepts/rules/basic-elements/>

<https://falco.org/docs/reference/rules/supported-fields/>

<https://falco.org/docs/reference/rules/default-macros/>

Create a Simple Falco Rule.

```
- rule: Detect curl Execution in Kubernetes Pod
  desc: Detects when the curl utility is executed within a Kubernetes pod.
  condition: >
    spawned_process and container and
    proc.name = "curl"
  output: >
    Suspicious process detected (curl) inside a Container.
  priority: WARNING
```

Testing

```
systemctl restart falco

kubectl run nginx-pod --image=nginx

kubectl exec -it nginx-pod -- bash

curl google.com
```

Improvement in Falco Rule Output

```
- rule: Detect curl Execution in Kubernetes Pod
  desc: Detects when the curl utility is executed within a Kubernetes pod.
  condition: >
    spawned_process and container and
    proc.name = "curl"
  output: >
    Suspicious process detected (curl) inside a container_id=%container.id and
    container_name=%container.name
  priority: WARNING
```

```
systemctl restart falco
```

Using Macros

```
- macro: sensitive_files
  condition: fd.name in (/tmp/sensitive.txt)

- rule: Access to Sensitive Files
  desc: Detect any process attempting to read or write sensitive files
  condition: sensitive_files
  output: "Sensitive file access detected (user=%user.name process=%proc.name
         file=%fd.name)"
  priority: WARNING
```

```
touch /tmp/sensitive.txt

systemctl restart falco
```

Optimized Macro Rule

```
- macro: sensitive_files
  condition: fd.name in (/tmp/sensitive.txt)

- rule: Access to Sensitive Files
  desc: Detect any process attempting to read or write sensitive files
  condition: open_read and sensitive_files
  output: "Sensitive file access detected (user=%user.name process=%proc.name
         file=%fd.name)"
  priority: WARNING
```

```
echo "Hi" /tmp/sensitive.txt
```

Falco Rule for /dev/mem

Verify /dev/mem access

```
kubectl run nginx-pod --image=nginx
kubectl exec -it nginx-pod -- bash
ls -l /dev

kubectl run privileged-pod --image=nginx --privileged
kubectl exec -it privileged-pod -- bash
ls -l /dev
```

Rule to Alert /dev/mem access from Container

```
- rule: Detect /dev/mem Access from Containers
  desc: Detect processes inside containers attempting to access /dev/mem
  condition: >
    (open_read or open_write) and
    fd.name=/dev/mem and container.id != host
  output: >
```

```
"Container: %container.id and %container.name attempted to access /dev/mem"
priority: CRITICAL
```

Testing

```
kubectl exec -it privileged-pod -- bash

cat /dev/mem
```

Find the Pod from Container ID

```
kubectl get pods -A -o json > pods.json

nano pods.json
```

Search using `CTRL+W` and add container ID

Falco Configuration File

Documentation Referred:

<https://github.com/falcosecurity/falco/blob/0.40.0/falco.yaml>

Commands Used for Testing

```
cd /etc/falco
cp falco.yaml falco.yaml.bak
```

```
nano falco.yaml
```

`CTRL+W` to search for `syslog_output` and replace true to false

```
syslog_output:
  enabled: false
```

```
systemctl restart falco
```

Start Falco manually

```
systemctl stop falco

falco

cat /etc/shadow
```

Revert the Changes

```
rm -f falco.yaml  
  
mv falco.yaml.bak falco.yaml  
  
systemctl restart falco  
  
cat /etc/shadow
```

Introduction to Sysdig

Documentation:

<https://github.com/draios/sysdig/wiki/Sysdig-User-Guide>

Install sysdig:

```
apt install sysdig
```

Commands Used:

```
sysdig  
sysdig proc.name=nano  
sysdig proc.name=cat or proc.name=nano
```

```
sysdig -l  
sysdig proc.name=cat and container.name!=host
```

```
csysdig
```

Sysdig Chisels:

```
sysdig -cl  
sysdig -c topprocs_cpu  
sysdig -c spectrogram  
sysdig -c spy_users  
sysdig -c ps
```

Audit Logging

Step 1 Create a directory for storing audit logs and audit policy:

```
cd /etc/kubernetes/  
mkdir audit
```

Step 2 - Mount the directory as HostPath Volumes in kube-apiserver:

```
- hostPath:
  path: /etc/kubernetes/audit
  type: DirectoryOrCreate
  name: auditing

- mountPath: /etc/kubernetes/audit
  name: auditing
```

Step 3 - Add Auditing Related Configuration in kube-apiserver:

```
- --audit-policy-file=/etc/kubernetes/audit/audit-policy.yaml
- --audit-log-path=/etc/kubernetes/audit/audit.log
```

Our Final Audit Policy:

```
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
- "RequestReceived"

rules:

- level: None
  resources:
  - group: ""
    resources: ["secrets"]
    namespaces: ["kube-system"]

- level: None
  users: ["system:kube-controller-manager"]
  resources:
  - group: ""
    resources: ["secrets"]

- level: RequestResponse
  resources:
  - group: ""
    resources: ["secrets"]
```

Reference - Falco Installation from Packages

Documentation Referenced:

<https://falco.org/docs/setup/packages/>

Install Falco for Ubuntu

```
curl -fsSL https://falco.org/repo/falcosecurity-packages.asc | \
sudo gpg --dearmor -o /usr/share/keyrings/falco-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/falco-archive-keyring.gpg] https://
download.falco.org/packages/deb stable main" | \
sudo tee -a /etc/apt/sources.list.d/falcosecurity.list
```

```
sudo apt-get update -y

sudo apt install -y dkms make linux-headers-$(uname -r)
sudo apt install -y clang llvm
sudo apt install -y dialog
```

```
sudo apt-get install -y falco
```

Check Falco status

```
systemctl status falco
```

Reference - Custom Falco Rule Examples

Documentation:

<https://falco.org/docs/rules/supported-fields/>

1st - Basic Rule:

```
- rule: The program "cat" is run in a container
  desc: An event will trigger every time you run cat in a container
  condition: evt.type = execve and container.id != host and proc.name = cat
  output: "cat was run inside a container"
  priority: INFO
```

2nd rule - Using Sysdig Field Class Elements:

```
- rule: The program "cat" is run in a container
  desc: An event will trigger every time you run cat in a container
  condition: evt.type = execve and container.id != host and proc.name = cat
  output: "cat was run inside a container (user=%username container=%container.name
         image=%container.image proc=%proc.cmdline)"
  priority: INFO
```

3rd rule - Using Macros and List:

```
- macro: custom_macro
  condition: evt.type = execve and container.id != host

- list: blacklist_binaries
  items: [cat, grep, date]

- rule: The program "cat" is run in a container
  desc: An event will trigger every time you run cat in a container
  condition: custom_macro and proc.name in (blacklist_binaries)
  output: "cat was run inside a container (user=%user.name container=%container.name
         image=%container.image proc=%proc.cmdline)"
  priority: INFO
```

Part 2: Add Output

```
- rule: The program "cat" is run in a container
  desc: An event will trigger every time you run cat in a container
  condition: evt.type = execve and container.id != host and proc.name = cat
  output: demo %evt.time %user.name %container.name
  priority: ERROR
  tags: [demo]
```

Reference - Falco from an Exam Perspective

Create a demo rule condition with the required output:

```
- rule: The program "cat" is run in a container
  desc: An event will trigger every time you run cat in a container
  condition: evt.type = execve and container.id != host and proc.name = cat
  output: demo %evt.time %user.name %proc.name
  priority: ERROR
  tags: [demo]
```

Dealing with Logging Use-Case:

```
timeout 25s falco | grep demo
nano /root/logs.txt
cat logs.txt | awk '{print $4 " " $5 " " $6}' > /tmp/logs.txt
cat /tmp/logs.txt
```